

IoT Systems

Riassunto Completo per l'Esame

Prof. Luca Bedogni — UNIMORE
A.A. 2024/2025

Francesco Caligiuri
Corso di Laurea Magistrale in Informatica

Parte I — Riassunto teoria del corso (dalle slide)
Parte II — Approfondimento tecnico del progetto di Vulnerability Assessment

Indice

1. Scenario IoT	7
1.1. Cos'è l'IoT	7
1.2. Architettura a tre livelli: Edge, Fog, Cloud	7
1.3. Reti e M2M	7
1.4. Sensori e miniaturizzazione	8
1.5. Microcontrollori e Smart Devices	8
1.6. Architettura di rete IoT	8
1.7. Collective Awareness	8
2. Use Cases	9
2.1. Monitoraggio Ambientale	9
2.1.1. Infrastructure-based vs Crowdsensing	9
2.2. Guida Autonoma	9
2.2.1. I 6 livelli SAE	9
2.2.2. Volumi dati (Intel)	10
2.3. Smart Home (Domotica)	10
2.4. Smart Factory	11
2.5. e-Health	11
2.6. Monitoraggio Strutturale	11
3. Ubiquitous Computing e Context-Aware Systems	12
3.1. Definizioni fondamentali	12
3.2. Context e Context-Awareness	12
3.2.1. Le quattro dimensioni del contesto	12
3.2.2. Contesto primario e secondario	13
3.2.3. Evoluzione storica della definizione	13
3.2.4. Tassonomia Intel 2008	13
3.2.5. Caratteristiche del contesto	13
3.2.6. Perché il Context Awareness è utile	14
3.3. Architettura di un sistema context-aware	14
3.4. Inferenza del contesto	14
3.5. Modellazione del Contesto	15
3.6. Active vs Passive Context Awareness	15
3.7. Positioning	15
3.8. Activity Recognition	16
3.8.1. Modello a cascata per transportation mode detection	16
3.8.2. Text-and-Drive: rilevamento distrazione alla guida	17
4. Boards e Hardware IoT	18
4.1. Microcontrollori vs Single-Board Computer	18
4.2. Arduino	18
4.2.1. Modello di programmazione	18
4.2.2. Arduino UNO (ATmega328P)	18
4.2.3. PWM (Pulse Width Modulation)	18
4.3. ESP32 / ESP8266	19
4.3.1. ESP8266	19
4.3.2. ESP32	19
4.4. Raspberry Pi	19
4.5. Interfacce di comunicazione	19
4.6. Sensori e Attuatori	20
5. LPWAN	21

5.1. Cos'è LPWAN	21
5.2. IEEE 802.15.4	21
5.2.1. Topologie di rete	21
5.2.2. Modalità MAC	21
5.3. LoRa e LoRaWAN	22
5.3.1. LoRa (livello fisico)	22
5.3.2. Spreading Factor (SF)	22
5.3.3. LoRaWAN (livello rete)	23
5.3.4. Classi di dispositivi	23
5.3.5. Sicurezza LoRaWAN	23
5.3.6. Attivazione: OTAA vs ABP	23
5.3.7. Efficienza energetica LoRa (motivi espliciti)	24
5.4. Sigfox	24
5.5. NB-IoT (Narrowband IoT)	24
5.6. Confronto LPWAN	25
6. Protocolli di Comunicazione	26
6.1. MQTT (Message Queuing Telemetry Transport)	26
6.1.1. Architettura	26
6.1.2. Topic	26
6.1.3. QoS (Quality of Service)	26
6.1.4. Funzionalità chiave	27
6.1.5. Formato messaggio	27
6.1.6. CONNECT packet fields	28
6.1.7. Codici di ritorno CONNACK	28
6.1.8. SUBACK return codes	28
6.1.9. MQTT 5 — novità principali	28
6.2. CoAP (Constrained Application Protocol)	28
6.2.1. Perché non HTTP?	28
6.2.2. Tipi di messaggio	29
6.2.3. Metodi e codici risposta	29
6.2.4. Header (4 byte fissi)	29
6.2.5. Meccanismi di affidabilità	29
6.2.6. Observe (RFC 7641)	30
6.2.7. Resource Discovery	30
6.2.8. Sicurezza	30
6.2.9. CoAP extensions	30
6.3. AMQP (Advanced Message Queuing Protocol)	31
6.4. BLE (Bluetooth Low Energy)	31
6.4.1. GAP (Generic Access Profile)	31
6.4.2. GATT (Generic Attribute Profile)	31
6.4.3. Parametri di connessione e sicurezza	32
6.5. Zigbee	32
6.5.1. Device types	32
6.5.2. Topologie	32
6.5.3. Profili applicativi e sicurezza	32
6.6. Confronto protocolli	32
6.7. Web of Things (WoT)	33
6.7.1. Problema che risolve	33
6.7.2. Thing Description (TD)	33
6.7.3. Interaction Affordances	33
6.7.4. Protocol Bindings	33

6.7.5. Semantic Interoperability	33
6.7.6. WoT Building Blocks	33
6.8. Formati di serializzazione dati	34
7. Task Offloading	35
7.1. Sensor Fusion	35
7.1.1. Pre-processing per la fusion	35
7.1.2. Architetture	35
7.2. Framework Concettuale dell'Offloading	35
7.2.1. Le 4 possibilità computazionali	36
7.2.2. Condizione energetica fondamentale	36
7.3. Offloading: Transform (Trasformazione del Segnale)	36
7.3.1. Tecniche su serie temporali	36
7.3.2. Tecniche su immagini	37
7.3.3. Sfide	37
7.4. Mobile Edge Computing (MEC)	37
7.5. Formulazione matematica	38
7.5.1. Calcolo locale	38
7.5.2. Offloading	38
7.5.3. Obiettivi di ottimizzazione	38
7.6. Multi-User MEC	38
7.7. Approcci risolutivi	39
8. ARIMA e Reinforcement Learning	40
8.1. Serie temporali	40
8.1.1. Stazionarietà	40
8.2. ACF e PACF	40
8.3. Modelli AR, MA, ARIMA	41
8.3.1. SARIMA	41
8.3.2. Box-Jenkins Methodology	41
8.4. Anomaly Detection	41
8.4.1. Rolling Standard Deviation	41
8.4.2. One-Class SVM	42
8.5. Reinforcement Learning	42
8.5.1. Elementi fondamentali	42
8.5.2. MDP (Markov Decision Process)	43
8.5.3. Value functions e Bellman	43
8.5.4. Q-Learning	43
8.5.5. Policy Gradient e Actor-Critic	44
8.5.6. Applicazioni RL nell'IoT	44
8.5.7. Esempio: Temperature Control (con Q-Learning)	44
9. AI on IoT	45
9.1. TinyML	45
9.2. Tecniche di compressione modelli	45
9.2.1. Quantizzazione	45
9.2.2. Pruning	46
9.2.3. Knowledge Distillation	46
9.3. Architetture efficienti	46
9.3.1. MobileNets	46
9.3.2. EfficientNet	46
9.4. Hardware per Edge AI	47
9.5. Software	47
9.6. Edge Impulse	47

9.6.1. CLI Tools	47
9.6.2. L'Impulso (pipeline)	47
9.6.3. Risultati del Training	48
9.6.4. Model Optimization: Quantizzazione	48
9.6.5. TFLite Workflow (dal modello al MCU)	48
9.7. Federated Learning	48
9.8. Split Computing	49
9.9. Early Exit	49
10. Node-RED	50
10.1. Cos'è Node-RED	50
10.1.1. Node-RED nell'architettura IoT	50
10.2. Concetti fondamentali	50
10.2.1. Nodi	50
10.2.2. Flows e Deploy	50
10.2.3. Messaggi e msg.payload	51
10.3. Nodi principali	51
10.4. Dashboard	51
10.5. Context Storage	51
11. Privacy nell'IoT	52
11.1. Il problema	52
11.2. Le 7 Privacy Threats (Bedogni)	52
11.3. Classificazione dati	53
11.4. Privacy by Design (7 principi Cavoukian)	53
11.5. GDPR e il contesto normativo europeo	54
11.6. PETs (Privacy Enhancing Technologies) — 8 strategie	54
11.7. Usable Privacy	54
11.8. OWASP IoT Top 10 (2018)	55
11.9. Cyber Resilience Act (EU 2024/2847)	55

Nota. Questo documento raccoglie la parte teorica del corso IoT Systems. L'approfondimento tecnico del vulnerability assessment svolto come progetto non è incluso: i finding sono oggetto di disclosure coordinata con i produttori, e una sintesi redatta per classi di vulnerabilità è disponibile in `security_assessment/`.

Scenario IoT

Cos'è l'IoT

L'**Internet of Things** nasce dall'intersezione di tre grandi famiglie tecnologiche: l'**ICT e la sua pervasività** (computer ovunque, sempre connessi), le **tecnologie di comunicazione** (reti wireless, protocolli leggeri), e l'**analisi dei dati** (machine learning, sistemi di supporto alle decisioni). Il risultato pratico è una rete di oggetti fisici dotati di sensori, software e connettività che possono scambiarsi informazioni tra loro e con sistemi remoti, senza necessariamente richiedere intervento umano.

L'idea di fondo è che i dispositivi non si limitino a raccogliere dati passivamente, ma siano capaci di **ragionare e agire**: un sensore di temperatura che misura e basta è un sensore qualunque; un sensore che legge la temperatura, la confronta con una soglia e accende il condizionatore è uno **Smart Device**. La differenza sta nella capacità di prendere decisioni autonome.

Architettura a tre livelli: Edge, Fog, Cloud

L'IoT non è una tecnologia piatta: i dispositivi, i dati e l'elaborazione sono distribuiti su tre livelli con caratteristiche e responsabilità molto diverse.

Edge — I miliardi di dispositivi fisici al bordo della rete (sensori, attuatori, microcontrollori). Sono risorse computazionali limitate, ma sono dove i dati nascono. Qui si fa la prima elaborazione: raccolta, pulizia e selezione dei dati rilevanti.

Fog — Milioni di nodi intermedi, più vicini allo scenario fisico rispetto al cloud. Aggregano dati da più dispositivi edge, fanno pre-elaborazione in tempo reale (ordine dei secondi) e riducono la quantità di dati da spedire al cloud. Fungono da ponte intelligente.

Cloud — Migliaia di data center con enorme potenza computazionale, scalabile on-demand. Lavorano su analisi a lungo termine (giorni, settimane, mesi), machine learning, storage massivo e distribuzione dei risultati.

Il flusso tipico in un sistema reale (es. smart building) è: i sensori raccolgono dati grezzi → il nodo edge pulisce e aggrega → il dato viaggia verso il fog per un'analisi in tempo reale → dal fog al cloud per big data analytics → i risultati (awareness) tornano verso il basso per guidare le azioni sui dispositivi. Ogni livello lavora a una granularità temporale diversa: l'edge reagisce in millisecondi, il fog in secondi, il cloud in ore o giorni.

Reti e M2M

L'IoT non usa un'unica tecnologia di rete: le reti sono **eterogenee** per natura, dai BAN (Body Area Network, i sensori indossabili a pochi centimetri dal corpo) fino alle WAN (Wide Area Network, comunicazione intercontinentale). Ogni classe di applicazione ha requisiti diversi di banda, energia e portata, e nessuna soluzione è universalmente ottimale.

Una caratteristica fondamentale che distingue l'IoT dal web tradizionale è che i peer della comunicazione **non sono necessariamente umani**: si parla di **Machine-to-Machine (M2M)** quando dispositivi si parlano direttamente, senza intermediazione umana. Quando questi dispositivi diventano accessibili tramite protocolli web standard (HTTP, REST) si parla di **Web of Things**.

Sensori e miniaturizzazione

Uno dei motori dell'esplosione IoT è la **miniaturizzazione** e il crollo dei costi dei sensori: molti sono ormai sotto il dollaro, grazie alle economie di scala dell'industria elettronica consumer. Uno smartphone moderno incorpora già una decina di sensori — accelerometro, giroscopio, magnetometro, barometro, prossimità, luce ambientale — e gli smartwatch ci aggiungono ECG, temperatura corporea e SpO2.

Questa pervasività ha anche una dimensione sociale: un sondaggio Forrester (2013, 4.657 adulti USA) ha chiesto dove le persone accetterebbero di indossare sensori sul corpo. Il 29% accetterebbe un clip sui vestiti, il 28% al polso, il 18% nella scarpa, il 15% integrati nei vestiti, fino al 3% che accetterebbe sensori tatuati sulla pelle. Il trend è chiaro: i sensori si avvicinano sempre di più al corpo.

Microcontrollori e Smart Devices

I **microcontrollori** moderni come l'ESP32 sono la colonna vertebrale dell'IoT edge: sono piccoli, consumano pochi milliwatt, costano pochi euro, integrano già WiFi e Bluetooth, e sono in grado di eseguire logica decisionale locale. Questo li rende diversi da un semplice sensore passivo: un ESP32 può leggere la temperatura, confrontarla con una soglia configurabile, e inviare un comando a un relè — tutto senza mai comunicare con il cloud.

La distinzione tra sensore e **Smart Device** è concettuale ma importante: un sensore produce dati, uno smart device elabora dati e produce **decisioni**. Quando più smart device comunicano tra loro e coordinano le proprie azioni, emergono capacità collettive che nessun singolo dispositivo potrebbe avere da solo.

Architettura di rete IoT

Un sistema IoT completo comprende quattro categorie di elementi che interagiscono:

- **Sensori (S)**: raccolgono misure dall'ambiente fisico (temperatura, umidità, pressione, ecc.)
- **Smart Devices (SD)**: raccolgono dati e prendono decisioni locali in base a regole o modelli
- **Attuatori (A)**: traducono le decisioni in azioni fisiche (accendere un motore, aprire una valvola)
- **Controller**: il cervello coordinatore della rete, fornisce regole, configurazione e logica di alto livello

Un sistema reale può avere più controller, ognuno responsabile di un dominio specifico (es. un controller per il controllo climatico, uno per la sicurezza). La separazione delle responsabilità è una scelta di design per limitare il blast radius di un guasto.

Collective Awareness

Il vero valore dell'IoT non sta nel singolo sensore, ma nella **visione collettiva** che emerge dalla combinazione di molti dati. Un termometro dice la temperatura di un punto; mille termometri distribuiti in una città permettono di capire le isole di calore urbane, pianificare la ventilazione degli edifici e ottimizzare i consumi energetici.

L'utente (o il sistema) definisce **obiettivi**; i dispositivi collaborano per raggiungerli nel modo migliore, dove «migliore» dipende dal contesto — può significare più veloce, più preciso, più efficiente energeticamente, o con il minor costo computazionale. Questa capacità di comprensione aggregata è chiamata **Collective Awareness**.

Use Cases

Monitoraggio Ambientale

Il caso d'uso più immediato dell'IoT è il **monitoraggio ambientale**: raccogliere misure (temperatura, umidità, rumore, qualità dell'aria, traffico) su un'area di interesse, in modo continuo e capillare. L'obiettivo è trasformare dati grezzi in conoscenza utile per decisioni — sia automatiche che umane.

Infrastructure-based vs Crowdsensing

Esistono due filosofie radicalmente diverse per raccogliere questi dati, ognuna con vantaggi e limiti opposti:

Infrastructure-based	Crowdsensing
Sensori fissi installati da un'organizzazione	Sfrutta i dispositivi già in possesso dei cittadini
Va progettata, costruita e mantenuta nel tempo	Non richiede infrastruttura dedicata
Vincolata ai task per cui è stata progettata	Flessibile: può essere attivata/disattivata a piacere
Precisione maggiore: sensori calibrati, installazione professionale	Meno precisa: più rumore, qualità variabile tra dispositivi

L'approccio **infrastructure-based** garantisce qualità e controllo, ma richiede investimenti significativi. Un esempio è **GLIDE Singapore**, il sistema del Land Transport Authority che controlla i semafori cittadini: usa sensori a induzione nel manto stradale e segnali dal traffico delle strade adiacenti per creare una «green wave», ovvero una sincronizzazione dei semafori che permette alle auto di attraversare più intersezioni consecutive senza fermarsi. I dati vengono dai sensori, le decisioni dagli operatori umani.

L'approccio **crowdsensing** sfrutta invece i milioni di smartphone già nelle tasche delle persone. **Google Traffic** è l'esempio più noto: i telefoni delle persone in auto trasmettono anonimamente la propria velocità; confrontando la velocità reale con quella storica attesa, il sistema deduce le condizioni del traffico in tempo reale. Funziona su scala globale senza installare un solo sensore fisico. La vulnerabilità di questo approccio è stata dimostrata in modo creativo dall'artista Simon Weckert, che nel 2020 ha trascinato 99 smartphone in un carretto attraverso le strade di Berlino, convincendo l'algoritmo di Google che ci fosse un ingorgo inesistente.

Una piattaforma più sofisticata è **SenSquare**, che combina dati di monitoraggio ambientale, traffico, trasporto pubblico, connettività e rumore, organizzandoli tramite il sistema MGRS (Military Grid Reference System) per strutturare le misurazioni gerarchicamente per zona geografica.

Le sfide del crowdsensing sono: come incentivare gli utenti a partecipare, come gestire la qualità variabile dei dati, come tutelare la privacy, e come fondere queste misure con dati da fonti eterogenee.

Guida Autonoma

La guida autonoma è uno dei campi applicativi più esigenti dell'IoT: i veicoli devono percepire l'ambiente in tempo reale, prendere decisioni di sicurezza in millisecondi, e comunicare con l'infrastruttura stradale e con gli altri veicoli.

I 6 livelli SAE

La Society of Automotive Engineers ha definito una scala a 6 livelli per descrivere il grado di autonomia di un veicolo:

Livello	Nome	Descrizione
0	Driver	Nessuna assistenza — il guidatore controlla tutto
1	Feet Off	Assistito (es. cruise control)
2	Hands Off	Parzialmente automatizzato (es. Tesla Autopilot su autostrada)
3	Eyes Off	Altamente automatizzato — il sistema guida, ma il guidatore deve essere pronto a riprendere
4	Mind Off	Completamente automatizzato in condizioni definite — il guidatore può dormire
5	Passenger	Autonomo completo — nessun volante necessario

I veicoli ai livelli 3–5 sono essenzialmente nodi IoT mobili: fondono dati da radar, sonar, telecamere, LiDAR e GPS per costruire una rappresentazione 3D dell’ambiente in tempo reale, e comunicano con l’infrastruttura stradale (V2I, Vehicle-to-Infrastructure) per informazioni su semafori, cantieri e incidenti.

Volumi dati (Intel)

La quantità di dati generata da un veicolo autonomo è impressionante e pone sfide enormi per trasmissione, storage e latenza:

Un veicolo autonomo genera circa **4.000 GB/giorno**:

- Radar: 10–100 KB/s
- Sonar: 10–100 KB/s
- GPS: 50 KB/s
- Telecamere: 20–40 MB/s
- LiDAR: 10–70 MB/s

La maggior parte di questi dati viene elaborata localmente, sul veicolo stesso, perché la **latenza è critica**: una decisione basata su dati arrivati con 100ms di ritardo può significare la differenza tra frenare e non frenare in tempo. Solo una piccola frazione viene trasmessa al cloud (per aggiornare le mappe, per training dei modelli, per telemetria).

Smart Home (Domotica)

La smart home è il caso d’uso più vicino alla vita quotidiana: una collezione di dispositivi (termostati, luci, serrature, elettrodomestici, sensori) che si coordinano per automatizzare compiti domestici e migliorare comfort, efficienza energetica e sicurezza. Il mercato è in forte crescita: da 815 milioni di dispositivi spediti nel 2019 a 1.396 milioni previsti nel 2023 secondo IDC.

Il problema principale della smart home non è tecnico ma di **interoperabilità**: ogni produttore usa protocolli e ecosistemi proprietari, creando le cosiddette **Isole di Things** (o Intranet of Things), dove i dispositivi parlano benissimo tra loro all’interno dello stesso brand ma non riescono a integrarsi con quelli di brand diversi. La soluzione passa per router multi-tecnologia (capaci di gestire Zigbee, LoRa, BLE e WiFi contemporaneamente), per l’omogenizzazione a livello dati, e per interfacce utente che nascondano questa complessità.

Home Assistant è la risposta open-source a questo problema: un controller centralizzato che integra centinaia di dispositivi COTS (Commercial Off-The-Shelf) e custom, permettendo di creare automazioni tra prodotti di brand diversi. È diventato uno standard di fatto per chi vuole una smart home aperta e non dipendente da un singolo vendor.

Tra gli strumenti citati nelle slide per la smart home automation figura anche **Routex**, un esempio di tool dedicato alla gestione e all'automazione dei dispositivi in ambienti «Islands of Things».

Le sfide irrisolte della smart home rimangono: interoperabilità tra ecosistemi, costo dei dispositivi, e soprattutto **privacy** (una casa piena di microfoni, telecamere e sensori che raccolgono dati sul comportamento domestico).

Smart Factory

La smart factory porta l'IoT nell'industria manifatturiera, con l'obiettivo di **digitalizzare l'intera catena produttiva**: sensori su ogni macchinario, controllo in tempo reale della qualità, automazione dei processi, manutenzione predittiva. Tecnologie come IoT, AI, Robotics, Cyber Physical Systems, Big Data e Cloud convergono in quello che viene chiamato Industria 4.0.

Un caso d'uso particolarmente importante è la **manutenzione predittiva**: invece di aspettare che un macchinario si guasti (manutenzione reattiva) o di sostituire componenti a intervalli fissi indipendentemente dalle condizioni reali (manutenzione preventiva), si monitorano continuamente vibrazioni, temperatura, consumo elettrico e altri indicatori per **predire** quando un guasto sta per avvenire. Un approccio comune è usare One-Class SVM per modellare il comportamento «normale» della macchina, così che qualsiasi deviazione significativa dal profilo normale venga segnalata come anomalia — senza bisogno di avere esempi di guasto etichettati, che spesso non esistono.

e-Health

Il settore sanitario beneficia enormemente della miniaturizzazione dei sensori: wearable come smartwatch e patch cutanee permettono il monitoraggio continuo di pazienti con patologie croniche (problemi cardiaci, diabete, demenza) senza costringerli a rimanere in ospedale. I dati raccolti (ECG, frequenza cardiaca, glicemia, attività) vengono trasmessi al medico o al sistema sanitario, che può intervenire prima che la situazione diventi critica.

Le sfide dell'e-health sono particolarmente delicate: la **privacy** dei dati medici è un diritto fondamentale; l'**efficienza della batteria** è critica (un sensore cardiaco che si scarica è un rischio); la **resilienza** è non negoziabile (il sistema non può avere downtime); e la **fiducia** degli utenti (pazienti e medici) deve essere guadagnata attraverso affidabilità e trasparenza.

Monitoraggio Strutturale

Il monitoraggio strutturale applica l'IoT alla salute degli edifici e delle infrastrutture: ponti, dighe, edifici storici e grattacieli vengono dotati di sensori di vibrazione, accelerometri, igrometri e sensori di deformazione per rilevare nel tempo segnali di degrado strutturale, resistenza sismica, infiltrazioni d'acqua. Sia sensori fissi (installati in punti critici della struttura) che sensori mobili su droni vengono impiegati per ispezioni periodiche o continue. L'obiettivo è anticipare guasti catastrofici — un crollo di ponte non è solo un danno economico ma un disastro umano.

Ubiquitous Computing e Context-Aware Systems

Definizioni fondamentali

Per capire l'IoT in profondità, occorre partire da alcune visioni teoriche che ne hanno anticipato i concetti chiave.

Ubiquitous Computing (Mark Weiser, Xerox PARC, 1991): Weiser aveva una visione radicale — «lo scopo di un computer è aiutarti a fare qualcos'altro». Il computer ideale diventa invisibile, si dissolve nell'ambiente, estende le capacità umane senza richiedere attenzione. L'**ubicomp** è l'uso collettivo di computer distribuiti nell'ambiente fisico, che collaborano in modo trasparente per l'utente.

Pervasive Computing: concetto strettamente correlato all'ubicomp, ma con enfasi sulla **mobilità**. È l'unione del mobile computing (smartphone, tablet) con computer statici presenti nell'ambiente, creando un continuum computazionale intorno all'utente.

Autonomic Computing (IBM, Paul Horn, 2001): sistemi capaci di gestirsi da soli, ispirandosi al sistema nervoso autonomo umano (che regola battito cardiaco e respirazione senza intervento cosciente). Le proprietà **Self-*** includono: self-healing (riparazione autonoma dei guasti), self-configuration (adattamento automatico), self-optimization (ottimizzazione continua delle prestazioni), self-protection (difesa dalle minacce).

Ambient Intelligence (Marzano & Aarts, Philips Research, 2003): ambienti dotati di interfacce intelligenti che **reagiscono alla presenza e alle intenzioni degli utenti**, anticipandone i bisogni. La tecnologia è al servizio dell'uomo, non viceversa.

Queste visioni convergono: un sistema IoT maturo è ubiquo (ovunque), pervasivo (su ogni dispositivo), autonomico (si gestisce da solo) e ambiently intelligent (risponde al contesto dell'utente).

Context e Context-Awareness

Il concetto di **contesto** è centrale in tutti questi paradigmi. La definizione più citata è quella di **Dey & Abowd (1999)**: il contesto è «qualsiasi informazione che può essere usata per caratterizzare la situazione di un'entità (persona, luogo, oggetto) rilevante per l'interazione utente-applicazione». In parole semplici: tutto ciò che un sistema sa della situazione corrente e che può influenzare il suo comportamento.

Un sistema **context-aware** usa queste informazioni per adattarsi: la stessa applicazione si comporta diversamente a seconda di dove sei, con chi sei, che ora è, e cosa stai facendo. Alcuni esempi concreti rendono l'idea meglio di qualsiasi definizione:

- La temperatura supera la soglia → il sistema accende automaticamente il raffreddamento
- C'è traffico sul percorso abituante → il navigatore suggerisce un'alternativa prima che tu parta
- L'utente è a Milano e cerca «Pizzeria» → i risultati mostrano pizzerie milanesi, non romane
- Nessuno è a casa secondo i sensori di presenza → il sistema attiva l'allarme

Le quattro dimensioni del contesto

Il contesto si organizza in quattro categorie tematiche:

Scenario	Ambientale	Computing	Preferenze utente
Calendario	Temperatura	Tecnologia rete	Configurazione
Posizione	Rumore	Costi batteria/rete	Comportamento
Tempo	Umidità	Bandwidth	Vicini
	Prossimità	Dispositivi rete	

Contesto primario e secondario

Non tutte le dimensioni del contesto hanno lo stesso peso. Il **contesto primario** comprende le quattro informazioni più fondamentali: **Where** (posizione), **Who** (identità), **When** (tempo), **What** (attività corrente). Sono il punto di partenza per qualsiasi ragionamento contestuale.

Il **contesto secondario** è derivato da quello primario per inferenza: conoscendo la posizione, si può derivare il meteo locale, i vicini nelle vicinanze, i servizi disponibili. Conoscendo l'attività, si può derivare il livello di attenzione disponibile. Il contesto secondario arricchisce la visione senza richiedere sensori aggiuntivi.

Evoluzione storica della definizione

- **1994 — Schilit, Adams, Want**: prima definizione formale — software che si adatta al **contesto computing** (hardware disponibile), **contesto utente** (posizione, persone vicine) e **contesto fisico** (illuminazione, rumore). Focus sulla reattività del software all'ambiente fisico.
- **2001 — Dey & Abowd**: definizione più generale e citata: «qualsiasi informazione che può essere usata per caratterizzare la situazione di un'entità (persona, luogo, oggetto) rilevante per l'interazione utente-applicazione, inclusa l'applicazione stessa.» Sposta il fuoco dall'hardware all'**interazione**.
- **2006 — ISO 9241-11**: prospettiva orientata all'usabilità — il contesto include goals e task degli utenti, l'infrastruttura tecnica, e l'ambiente fisico e sociale. Utile per il design di interfacce.
- **2008 — Intel**: tassonomia pragmatica a 3 categorie per il design di sistemi concreti (vedi sotto).

Tassonomia Intel 2008

- **Environmental Context**: ciò che circonda l'utente — ambiente fisico (temperatura, luce, rumore), dispositivi nelle vicinanze, servizi disponibili nella rete locale
- **User Context**: tutto sull'utente — identità, stato fisico (postura, movimento), stato fisiologico (frequenza cardiaca, stress), attività correnti, contatti, preferenze
- **Platform Context**: lo stato della piattaforma computazionale — posizione GPS, movimento, connettività di rete, livello batteria, funzionalità disponibili, app in esecuzione, memoria disponibile

Caratteristiche del contesto

Tre proprietà fondamentali del contesto che ogni sistema context-aware deve gestire:

Imperfetto: i sensori non sono perfetti — hanno rumore, si guastano, devono essere calibrati, possono essere temporaneamente non disponibili. Non si può progettare un sistema che assume un contesto perfetto; la robustezza all'incertezza è un requisito di progetto, non un'opzione.

Multiple rappresentazioni: lo stesso dato contestuale può essere rappresentato in modo diverso — come semplice coppia key-value (`location=Milan`), come struttura XML gerarchica, come logica

formale, come ontologia semantica. La scelta rappresenta un tradeoff tra semplicità e espressività: più il modello è ricco, più è costoso da mantenere e interrogare.

Correlato: il contesto primario **implica** contesto secondario. La correlazione può essere sfruttata per inferire informazioni nuove senza sensori aggiuntivi (es. posizione → meteo → abbigliamento consigliato), ma richiede attenzione perché le correlazioni possono essere false positivi.

Perché il Context Awareness è utile

Il context awareness migliora un sistema lungo quattro dimensioni:

1. **Acquisizione:** il contesto guida **cosa raccogliere** — non ha senso tenere tutti i sensori attivi sempre; il contesto decide quando campionare e a quale frequenza, risparmiando energia
2. **Organizzazione:** le informazioni vengono strutturate in modo che siano facili da interrogare per il contesto corrente, anziché come un blob di dati grezzi
3. **Discriminazione:** di tutte le informazioni disponibili, solo quelle rilevanti per la situazione corrente vengono presentate o usate, riducendo il rumore cognitivo e computazionale
4. **Adattamento:** il comportamento del sistema cambia in risposta al contesto — ridurre la frequenza di sampling quando l'utente è fermo, aumentarla quando si muove velocemente

Architettura di un sistema context-aware

Un sistema context-aware si articola su tre funzionalità principali che formano un loop continuo: **Sensing** (i sensori raccolgono dati dall'ambiente fisico e dall'utente) → **Computing/Thinking** (i dati vengono processati, il contesto viene inferito, le decisioni vengono prese) → **Acting** (il sistema modifica il suo comportamento o l'ambiente). Questo ciclo non si ferma mai: ogni azione genera nuovi dati contestuali.

Architetturalmente, il sistema è a strati: in basso il **Sensing Layer** (sensori fisici), in mezzo il **Data Storage Layer** (computing e storage), in alto l'**Acting Layer** (applicazioni e servizi che usano il contesto). La pipeline dettagliata è: Raw Data → Pre-processing (pulizia, normalizzazione, allineamento temporale) → Modeling (inferenza del contesto) → App/Service, con un loop di feedback dal Ground Truth per validare e migliorare il modello nel tempo.

Inferenza del contesto

Il cuore di un sistema context-aware è l'**inferenza**: dato un insieme di misure grezze, come si arriva a un'interpretazione ad alto livello (es. «l'utente sta guidando in autostrada»)?

Due approcci fondamentali, spesso usati insieme:

Il **Machine Learning** tratta l'inferenza del contesto come un problema di classificazione: si raccoglie un dataset etichettato (misure + ground truth del contesto), si addestra un modello, e si usa in inferenza. Funziona bene per contesti complessi con pattern non ovvi (es. activity recognition da accelerometro su smartphone). Richiede dati di training di qualità.

Le **Regole esplicite** codificano la conoscenza del dominio direttamente: «se la velocità GPS supera 50 km/h, l'utente è in auto o in treno». Sono semplici, interpretabili, e non richiedono training data, ma non scalano bene a contesti complessi e richiedono manutenzione manuale.

Il **context layering** è un approccio ibrido che raffina progressivamente il contesto aggiungendo vincoli a strati:

- Layer 1: Temperatura < 28° → condizioni potenzialmente adatte per correre
- Layer 2: + Umidità < 80% → condizioni effettivamente confortevoli
- Layer 3: + PM10 basso → qualità aria accettabile
- Layer 4: + Ultima corsa > 2 giorni fa → l'utente probabilmente vuole uscire

Ogni strato aggiunge precisione ma richiede dati aggiuntivi. Gli algoritmi devono gestire la **graceful degradation**: se un layer di dati non è disponibile (sensore guasto, GPS assente), il sistema non deve bloccarsi ma degradare in modo elegante con un contesto meno preciso.

Modellazione del Contesto

Come si rappresenta formalmente il contesto nel sistema? Esistono cinque approcci con diversi tradeoff espressività/complessità:

- **Key-Value**: il più semplice (`temperature=22`, `location=office`). Facile da implementare e interrogare. Non esprime struttura, relazioni tra dati, o semantica. Adatto per sistemi semplici.
- **Markup (XML/JSON)**: struttura gerarchica leggibile da macchina. Standard, interoperabile, ben supportato da librerie. Verboso, semantica limitata — si sa la struttura dei dati ma non il loro significato.
- **Graphical (UML/ORM)**: modellazione visuale orientata al design. Utile per comunicare l'architettura del sistema, ma non direttamente eseguibile — deve essere tradotto in un'altra forma per l'implementazione.
- **Logic (Prolog, Description Logic)**: regole formali con ragionamento automatico. Permette di derivare fatti nuovi da quelli noti. La scalabilità è limitata: con molte regole e fatti, il ragionamento diventa lento.
- **Ontology (OWL, RDF)**: grafi di conoscenza con assiomi semantici ricchi. Permettono ragionamento sofisticato, riuso tra sistemi, e interoperabilità semantica. Alta complessità di design e query. Standard rilevanti: SOUPA, COBRA-ONT.

La scelta del modello dipende dal contesto applicativo: sistemi embedded semplici usano key-value; sistemi enterprise che devono integrare dati eterogenei usano ontologie.

Active vs Passive Context Awareness

Una distinzione importante è tra l'uso **attivo** e **passivo** del contesto.

In modalità **active**, il contesto inferito **cambia direttamente il comportamento dell'applicazione**: il navigatore ricalcola il percorso senza che l'utente lo chieda, il condizionatore si accende quando la temperatura supera la soglia, la casa attiva l'allarme quando non rileva presenze. Il sistema prende decisioni autonomamente.

In modalità **passive**, il contesto **arricchisce l'esperienza informativa dell'utente** senza prendere decisioni al suo posto: il telefono mostra una notifica su un ingorgo sulla route abituale, ma non cambia la destinazione da solo; il meteo consigliato appare nella schermata di lock, ma l'utente decide se prendere l'ombrello. Il controllo rimane all'utente.

La scelta tra i due dipende dal livello di fiducia che l'utente e il sistema ripongono nell'accuratezza del contesto inferito, e dalla criticità delle decisioni.

Positioning

La posizione è il dato contestuale più usato (e più utile). Esistono tre famiglie di tecnologie di localizzazione, ognuna ottimale in scenari diversi:

- **Satellite-based** (GPS, Galileo, GLONASS): basato sul Time of Arrival di segnali da satelliti. Precisione dell'ordine del metro, copertura mondiale. Richiede Line of Sight verso i satelliti — non funziona

indoor o in aree urbane dense con effetto canyon. Alto consumo energetico (critico per dispositivi IoT a batteria).

- **Network-based** (GSM, WiFi fingerprinting, Cellular): usa l'RSSI delle celle o degli access point per trilateration o fingerprinting. Funziona in ambienti NLOS (Non Line of Sight), precisione da decine di metri a centinaia di metri. Non richiede hardware dedicato (usa la connettività già presente), ma introduce overhead di comunicazione.
- **Indoor** (IR, RFID, BLE, WLAN): tecnologie a corto raggio per localizzazione in ambienti chiusi. BLE beacon può arrivare a precisione di 1–3m; RFID permette identificazione precisa ma a contatto/cortissimo raggio. Richiede installazione di infrastruttura (beacon, tag) nell'edificio.

Activity Recognition

Il **riconoscimento delle attività** è uno dei problemi più studiati nel context-aware computing: dato un flusso di dati da sensori (accelerometro, giroscopio, GPS, microfono), classificare l'attività corrente dell'utente. Le attività spaziano molto: attività domestiche (cucinare, dormire, riposare), mobilità (camminare, correre, guidare, andare in bici, essere in treno), attività sportive, attività lavorative. Il paradigma base è sempre SENSE → PROCESS → ACT.

Per il **transportation mode detection** (capire se l'utente è a piedi, in auto, in treno) si usano principalmente velocità media (da GPS) e pattern dell'accelerometro. La «firma» dell'accelerometro sui tre assi X, Y, Z è molto diversa per diverse modalità di trasporto: camminare produce oscillazioni ritmiche a 1–2 Hz con ampiezza moderata; stare in auto produce vibrazioni di alta frequenza ma bassa ampiezza; il treno ha vibrazioni caratteristiche legate alle rotaie.

Non tutti i sensori e utenti sono uguali: lo stesso accelerometro produce letture diverse a seconda di dove il telefono è posizionato (tasca, borsa, mano). E utenti diversi hanno stili di camminata diversi — un modello addestrato su una persona può non generalizzare bene ad altre. Per questo si preferiscono modelli adattativi che si personalizzano nel tempo.

L'approccio metodologico corretto per affrontare un problema di activity recognition è: partire da un **need** chiaro (cosa si vuole riconoscere e perché), identificare i **dati** utili per distinguere le attività, progettare come **raccoglierli** (quale sensore, a quale frequenza, con quale preprocessing), **analizzare** i dati per derivare feature discriminanti e costruire un modello, e infine **deployare** il modello sul dispositivo target.

Modello a cascata per transportation mode detection

Un singolo classificatore soffre di alta confusione tra modalità simili (es. bus vs treno). Le slide propongono un'architettura **a cascata a 3 livelli** per ridurre l'errore complessivo:

1. **Livello coarse** — Random Forest su features GPS (velocità media, deviazione standard): separa **motorized** (car/bus/train) da **non-motorized** (walk/bike). La divisione è netta perché le velocità medie sono molto diverse.
2. **Livello refined** — Random Tree: dentro ogni macro-classe, distingue le modalità specifiche (bus/car/train nel ramo motorized; walk/bike nel ramo non-motorized). Ogni classificatore lavora su un sottoinsieme più omogeneo di classi, quindi ha meno errori.
3. **Livello temporale** — Decision Tree + **history module**: sfrutta le transizioni temporali (le modalità di trasporto cambiano lentamente nella realtà) per correggere classificazioni anomale. Se il sistema classifica «treno → bici → treno», il modulo temporale forzala a «treno continuo» perché quella transizione non è fisicamente plausibile.

Il risultato è un errore di classificazione complessivo $< 5\%$, molto inferiore a quello ottenibile con i singoli classificatori separati.

Text-and-Drive: rilevamento distrazione alla guida

Caso studio di context-aware computing passivo: lo smartphone sul cruscotto rileva se il guidatore sta usando il telefono analizzando i dati dell'accelerometro. La digitazione genera pattern di vibrazione distinguibili dalla vibrazione del veicolo in movimento. Feature extraction su finestre temporali + classificatore binario (distracted / not-distracted). L'aspetto chiave è che il sensore è già presente sul dispositivo e non richiede hardware aggiuntivo — è un esempio di context awareness **non invasivo** con sensori passivi.

Boards e Hardware IoT

Microcontrollori vs Single-Board Computer

L'hardware IoT si divide in due grandi famiglie con caratteristiche complementari. La scelta tra le due dipende interamente dai requisiti dell'applicazione.

Microcontrollore	Single-Board Computer
Meno costoso (pochi euro)	Più potente ma più costoso
Esegue un singolo programma in loop	Supporta multithreading e processi concorrenti
Molto efficiente energeticamente (mA o meno)	Consumo più elevato (centinaia di mA)
Solido e affidabile: nessun OS che può crashare	Supporta OS completo (Linux)
Ideale per task singoli e periodici	Ideale per operazioni computazionalmente intensive

I **microcontrollori** sono la scelta naturale per i nodi IoT al bordo della rete: controllano sensori, leggono dati, applicano logica semplice, trasmettono. I **Single-Board Computer** (SBC) sono più adatti al livello fog/edge intermedio, dove serve aggregare dati da più nodi, fare preprocessing più sofisticato, o eseguire modelli ML.

Arduino

Modello di programmazione

Arduino ha un modello di programmazione intenzionalmente semplice: due funzioni obbligatorie, `setup()` che viene eseguita **una sola volta** all'avvio per inizializzare pin, periferiche e variabili, e `loop()` che viene eseguita in **loop infinito** finché il dispositivo è alimentato. Non c'è scheduler, non c'è multithreading: il programma è sequenziale. Questo lo rende molto prevedibile e adatto a task di controllo real-time.

Le funzioni più usate nel laboratorio IoT: `digitalRead(pin)` / `digitalWrite(pin, value)` per leggere e scrivere segnali digitali (HIGH/LOW), `analogRead(pin)` / `analogWrite(pin, value)` per la gestione analogica, `Serial.begin(baudRate)` per aprire la comunicazione seriale (essenziale per il debugging), e `delay(ms)` per pause temporizzate.

Arduino UNO (ATmega328P)

L'UNO è il modello di riferimento: microcontrollore ATmega328P a 5V, clock 16 MHz, 14 pin digitali (6 dei quali supportano PWM), 6 pin analogici, 32 KB di Flash per il programma, 2 KB di SRAM per le variabili a runtime, 1 KB di EEPROM per dati persistenti. L'ADC a 10 bit mappa la tensione 0–5V in un valore intero nel range [0:1023] — con una risoluzione di circa 4.9 mV per step.

PWM (Pulse Width Modulation)

I pin contrassegnati con `~` sull'UNO supportano PWM. La funzione `analogWrite(pin, value)` accetta un valore da 0 a 255 che corrisponde al **duty cycle**: la percentuale del tempo in cui il pin è HIGH. È importante capire che il PWM **non produce una vera tensione analogica**: produce invece un segnale digitale che oscilla rapidamente tra 0V e 5V. La media temporale di questa oscillazione

simula il comportamento di una tensione analogica — un motore DC o un LED percepiscono la media, non il singolo ciclo. `analogWrite(0)` = 0% duty cycle (sempre LOW), `analogWrite(127)` $\approx$ 50%, `analogWrite(255)` = 100% (sempre HIGH).

ESP32 / ESP8266

La famiglia ESP di Espressif ha rivoluzionato il mercato IoT portando connettività WiFi a costo bassissimo su chip di dimensioni ridottissime.

ESP8266

Il più semplice dei due: WiFi 802.11 b/g/n integrato, 17 GPIO, un solo canale ADC a 10 bit, alimentazione a 3.3V, clock 80 MHz, circa 4 MB Flash e 80 KB RAM. Essenzialmente è un chip WiFi con un microcontrollore integrato e uno stack TCP/IP completo. Sufficiente per applicazioni IoT semplici che richiedono connettività WiFi a basso costo.

ESP32

L'ESP32 è molto più capace: **dual-core** Xtensa LX6 fino a 240 MHz, WiFi e Bluetooth 4.2 (sia BR/EDR classico che BLE), 34 GPIO, 18 canali ADC a 12 bit (risoluzione 4096 livelli), 2 DAC a 8 bit per output analogico reale, 520 KB di SRAM, fino a 16 MB Flash esterna. Include anche sensori capacitivi touch e un sensore Hall effect. Supporta SPI, I2C, UART, I2S (audio), CAN. È la piattaforma di riferimento per prototipi IoT avanzati. Sia ESP32 che ESP8266 possono essere programmati con l'Arduino IDE aggiungendo le URL delle board nelle Preferenze.

Raspberry Pi

Il Raspberry Pi è lo SBC di riferimento per l'IoT: esegue un sistema operativo Linux completo, supporta Python, C, Java, multithreading, database, networking avanzato. Il Pi 4 monta un quad-core ARM Cortex-A72 a 1.5 GHz, fino a 8 GB di RAM, WiFi dual-band, Bluetooth 5.0, Gigabit Ethernet, 2 porte USB 3.0, 2 uscite micro-HDMI 4K e 40 pin GPIO per l'interfacciamento con hardware esterno. Con il Pi è possibile fare cose impossibili su un microcontrollore: eseguire un server MQTT, fare inferenza con TensorFlow, gestire un database SQLite, servire una dashboard web.

Interfacce di comunicazione

Per collegare sensori e attuatori a microcontrollori e SBC si usano tre interfacce standard, ognuna con un caso d'uso prevalente:

Caratteristica	I2C	SPI	UART
Fili	2 (SDA, SCL)	4+ (MOSI, MISO, SCK, SS)	2 (TX, RX)
Duplex	Half	Full	Full
Velocità	Fino a 3.4 Mbps	Fino a 10+ Mbps	Fino a 115.2 kbps
Dispositivi	Multipli (indirizzamento)	Multipli (chip select)	Punto-punto
Clock	Sincrono	Sincrono	Asincrono

I2C è la scelta naturale per connettere **molte sensori con pochi fili**: usa solo due linee (SDA per i dati, SCL per il clock) e supporta fino a 128 dispositivi sullo stesso bus grazie all'indirizzamento a 7 bit.

Opera in half-duplex (una direzione alla volta) e richiede resistenze pull-up sulle linee. Ideale per sensori che non richiedono alta velocità (temperatura, umidità, pressione, display OLED).

SPI è più veloce e full-duplex (può trasmettere e ricevere contemporaneamente), ma richiede un filo SS (Slave Select) dedicato per ogni dispositivo. Non ha meccanismo di ACK — il master non sa se lo slave ha ricevuto i dati. Ideale per display, schede SD, e sensori ad alta frequenza di campionamento.

UART è l'interfaccia **asincrona punto-a-punto**: nessun clock condiviso, i due lati si accordano a priori sul baud rate. Formato del frame: Start bit (sempre) + 5–9 Data bits + Parity bit opzionale + 1–2 Stop bit. È l'interfaccia usata per la comunicazione seriale con il PC (debug), per moduli GPS, e per comunicazione tra microcontrollori.

Sensori e Attuatori

I **sensori** più comuni nei laboratori IoT: temperatura e umidità con DHT11 (economico, precisione $\pm 2^\circ\text{C}$) o DHT22 (più preciso, $\pm 0.5^\circ\text{C}$), sensori di luce LDR (fotoresistore analogico) o BH1750 (I2C, lux calibrati), movimento con PIR (rilevamento infrarosso passivo) o HC-SR04 (ultrasuoni, distanza), pressione barometrica BMP280 (I2C, utile anche per altitudine), gas MQ-2 (fumo, GPL) o MQ-135 (aria, CO2), e il MPU6050 che integra accelerometro e giroscopio a 3 assi su I2C.

Gli **attuatori** più usati: LED (uscita digitale), motori DC (richiedono driver come L298N), servomotori (controllo angolare via PWM), motori stepper (controllo preciso di posizione), buzzer (output sonoro), relè (commutazione di carichi ad alta tensione), display LCD o OLED (output visivo).

LPWAN

Cos'è LPWAN

Le reti LPWAN (Low Power Wide Area Network) nascono da un'esigenza pratica che le reti tradizionali non riescono a soddisfare: connettere milioni di piccoli sensori distribuiti su vaste aree geografiche, alimentati a batteria per anni, che trasmettono pochi byte di dati con bassa frequenza. WiFi è troppo energivoro e ha portata limitata; le reti cellulari 4G/LTE consumano troppo e costano troppo per questo tipo di traffico; Zigbee e Bluetooth hanno portata insufficiente.

Le LPWAN risolvono questo problema accettando un tradeoff esplicito: **rinunciare a banda (0.3–50 kbps) e latenza (secondi, non millisecondi) in cambio di portata (2–15+ km in ambiente urbano, 40+ km in rurale) e anni di autonomia a batteria**. Questo le rende ideali per contatori intelligenti, monitoraggio ambientale, asset tracking, sensori agricoli — tutte applicazioni dove i dati cambiano lentamente e non richiedono risposta immediata.

IEEE 802.15.4

IEEE 802.15.4 è lo standard di livello fisico e MAC su cui si appoggiano molti protocolli IoT a corto raggio (Zigbee, Thread, 6LoWPAN). Definisce le bande operative — 2.4 GHz a livello mondiale (250 kbps), 868 MHz in Europa (20 kbps), 915 MHz nelle Americhe (40 kbps) — e due classi di dispositivi con capacità molto diverse.

I **FFD** (Full Function Device) possono assumere qualsiasi ruolo nella rete (coordinatore, router, dispositivo finale), supportano tutte le topologie e possono comunicare con qualsiasi altro nodo. I **RFD** (Reduced Function Device) sono più semplici e limitati: supportano solo la topologia a stella e comunicano esclusivamente con il coordinatore. Questa distinzione permette di avere nodi RFD molto economici e a basso consumo per i ruoli più semplici (foglie della rete).

Topologie di rete

La rete può organizzarsi in tre topologie:

- **Star**: tutti i device comunicano con un unico coordinatore centrale. Semplice e robusta, ma il coordinatore è un single point of failure.
- **Ad-Hoc (Peer-to-Peer)**: qualsiasi device può comunicare direttamente con qualsiasi altro. Massima flessibilità, ma richiede che tutti i nodi siano FFD.
- **Combined (Clustered stars)**: più topologie a stella collegate tra loro tramite i coordinatori. Gerarchica e scalabile, permette di coprire aree più grandi.

Modalità MAC

Lo standard 802.15.4 definisce quattro modalità di accesso al canale che permettono di ottimizzare il consumo energetico a seconda del pattern di traffico:

- **Beacon mode**: il coordinatore trasmette periodicamente un **beacon frame** di sincronizzazione. I device si svegliano solo in corrispondenza del beacon per verificare se ci sono dati, poi tornano a dormire. Il risparmio energetico è notevole perché il device non deve restare in ascolto continuamente.

- **Contention mode (CSMA/CA)**: accesso competitivo al canale — prima di trasmettere, il device ascolta il canale; se libero, trasmette; se occupato, applica un backoff esponenziale casuale e riprova. Possibili collisioni, ma staticamente poco probabili se il traffico è basso.
- **Contention-free (GTS — Guaranteed Time Slots)**: il coordinatore assegna slot temporali dedicati a device specifici nella superframe. Non ci sono collisioni, la latenza è deterministica. Ideale per sensori con traffico periodico e requisiti real-time.
- **Battery-saving mode**: il device dorme quasi sempre, si sveglia solo al beacon per verificare se il coordinatore ha dati in attesa (bit di pending nel beacon). Se non ci sono dati, torna immediatamente a dormire.

LoRa e LoRaWAN

LoRa e LoRaWAN sono due cose distinte che operano a livelli diversi dello stack: LoRa è la tecnologia di modulazione radio (livello fisico), LoRaWAN è il protocollo di rete (livello MAC e rete) costruito sopra LoRa.

LoRa (livello fisico)

LoRa usa la modulazione **Chirp Spread Spectrum (CSS)**. Un **chirp** è un segnale radio la cui frequenza varia nel tempo: sale linealmente da un valore minimo a uno massimo (**up-chirp**) o scende (**down-chirp**). Il simbolo LoRa è codificato come uno shift di fase nell'andamento del chirp. Questa modulazione ha proprietà eccezionali per ambienti difficili: è molto robusta a interferenze narrowband (un'interferenza su una frequenza specifica disturba solo una piccola porzione del chirp), al multipath (segnali riflessi arrivano sfasati ma il ricevitore riesce a distinguerli), e all'effetto Doppler (utile per dispositivi in movimento).

LoRa opera su bande ISM non licenziate: 868 MHz in Europa, 915 MHz nelle Americhe, 433 MHz in Asia. Bande non licenziate significa che non serve pagare per lo spettro, ma ci sono vincoli normativi sul duty cycle (in Europa, massimo 1% del tempo di trasmissione).

Spreading Factor (SF)

Il parametro più importante di LoRa è lo **Spreading Factor**, che va da SF7 a SF12. Aumentare l'SF ha due effetti opposti: **aumenta il range** (perché il segnale è più facile da estrarre dal rumore — vedi la sensibilità nella tabella) ma **riduce il data rate** e aumenta il tempo di trasmissione (airtime). Ogni step di SF raddoppia circa l'airtime: un pacchetto trasmesso a SF12 è $32\times$ più lento che a SF7.

La caratteristica più importante degli SF in LoRa è che **SF diversi sono ortogonali tra loro**: un gateway può ricevere simultaneamente trasmissioni a SF7, SF8, SF9, SF10, SF11 e SF12 senza che si interferiscano. Questo è il motivo per cui LoRa scala bene: tanti device con SF diversi possono trasmettere contemporaneamente.

SF	BW (kHz)	Bit Rate (bps)	Sensibilità (dBm)
7	125	5.470	-123
8	125	3.125	-126
9	125	1.760	-129
10	125	980	-132
11	125	440	-134.5
12	125	290	-137

La **bandwidth** (125, 250 o 500 kHz) influenza il data rate: bandwidth maggiore = più veloce ma meno sensibile (il rumore in banda cresce). Il **Coding Rate** (4/5, 4/6, 4/7, 4/8) aggiunge ridondanza per la correzione degli errori: 4/5 significa che su 5 bit trasmessi, 4 sono dati e 1 è ridondanza.

LoRaWAN (livello rete)

LoRaWAN definisce l'architettura di rete sopra LoRa. La topologia è **star-of-stars**: i dispositivi finali (**End Devices**) comunicano via radio con uno o più **Gateways**, i gateway inoltrano i pacchetti al **Network Server** via connessione IP standard, e il Network Server li consegna all'**Application Server** dell'applicazione finale. I dispositivi **non comunicano tra loro** — non esiste routing multi-hop in LoRaWAN; tutta la comunicazione passa dai gateway. Questo semplifica enormemente i dispositivi finali (nessuna routing table, nessun overhead di rete) ed è uno dei motivi principali dell'eccellente efficienza energetica.

Classi di dispositivi

Il tradeoff tra consumo e latenza downlink è gestito attraverso tre classi di dispositivi:

Classe	Descrizione	Consumo	Latenza DL
A	Uplink-initiated. Il device trasmette quando ha dati, poi apre 2 finestre RX (RX1 dopo 1s, RX2 dopo 2s dalla fine della TX). Il network server può inviare downlink solo in queste finestre.	Minimo	Alta
B	Tutto ciò che fa Classe A, più finestre RX aggiuntive schedate tramite beacon periodico dal gateway (ping slot). Il server conosce quando il device è in ascolto.	Medio	Media
C	Il device è sempre in ascolto (finestra RX aperta continuamente, tranne durante la propria trasmissione). Latenza downlink quasi nulla.	Massimo	Minima

La Classe A è la scelta per la stragrande maggioranza dei sensori IoT (contatori, ambientali, agricoltura): trasmettono raramente e non hanno bisogno di ricevere comandi in tempo reale. La Classe C è per attuatori o dispositivi collegati alla rete elettrica.

Sicurezza LoRaWAN

LoRaWAN usa crittografia **AES-128** a due livelli indipendenti, una scelta architetturale importante: il network server e l'application server possono essere gestiti da entità diverse, e ognuno vede solo ciò che gli compete.

- **NwkSKey** (Network Session Key): usata tra device e network server per proteggere il livello MAC e calcolare il MIC (Message Integrity Code) che garantisce autenticità e integrità
- **AppSKey** (Application Session Key): usata tra device e application server per cifrare il payload applicativo end-to-end — il network server vede i metadati ma **non** il contenuto del messaggio
- **AppKey**: chiave master a lungo termine, usata solo durante la procedura di join OTAA per derivare in modo sicuro le due session key

Attivazione: OTAA vs ABP

Come vengono configurate le chiavi sul dispositivo? Esistono due modalità:

OTAA (Over-The-Air Activation) è l'approccio sicuro e raccomandato: il device invia un Join Request contenente il suo DevEUI (identificatore univoco del dispositivo), l'AppEUI (identificatore dell'applicazione) e un DevNonce casuale. Il network server risponde con un Join Accept. Entrambi i lati derivano le session key (NwkSKey e AppSKey) dall'AppKey combinata con i nonce — senza mai

trasmettere le chiavi stesse. Le chiavi possono essere rigenerate ad ogni connessione, il che limita l'impatto di un'eventuale compromissione.

ABP (Activation By Personalization) è l'approccio semplificato: DevAddr, NwkSKey e AppSKey sono hard-coded nel firmware del device prima del deployment. Non c'è procedura di join — il device è pronto a trasmettere immediatamente. Più semplice ma meno sicuro: le chiavi sono fisse per tutta la vita del dispositivo.

ADR (Adaptive Data Rate): il network server monitora la qualità dei segnali ricevuti da ogni device e aggiusta dinamicamente SF, bandwidth e potenza di trasmissione per trovare il tradeoff ottimale tra portata e durata della batteria per ogni singolo device.

Efficienza energetica LoRa (motivi espliciti)

LoRa permette **anni di batteria** grazie a tre fattori strutturali che si sommano:

1. **Nessun routing**: la topologia star-of-stars elimina completamente l'overhead di routing — nessuna tabella da mantenere, nessun beacon da ascoltare, nessun neighbour discovery
2. **Duty cycle bassissimo**: i device trasmettono solo l'**1% del tempo** (vincolo regolatorio europeo sulle bande ISM 868 MHz) — significa che per ogni secondo di trasmissione, il device può dormire 99 secondi
3. **MAC layer ottimizzato per Classe A**: il device si sveglia solo per trasmettere e per le due brevissime finestre RX subito dopo. Il resto del tempo dorme. Il consumo in sleep può essere di pochi microampere.

Sigfox

Sigfox è un'alternativa proprietaria alle LPWAN aperte, basata sulla tecnologia **Ultra-Narrowband (UNB)**: i messaggi occupano una larghezza di banda di soli 100 Hz (confronto: LoRa usa 125 kHz). Questa estrema narrowband permette una sensibilità del ricevitore eccezionale e quindi range molto lunghi (fino a 30–50 km in rurale), ma limita severamente il throughput.

Il modello di business è «network-as-a-service»: Sigfox gestisce tutta l'infrastruttura radio (base station) e l'utente paga per dispositivo. Questo riduce la complessità operativa ma crea dipendenza da un singolo vendor.

I vincoli sono severi: uplink massimo 140 messaggi/giorno con payload di 12 byte; downlink massimo 4 messaggi/giorno con 8 byte. Ogni messaggio viene trasmesso **3 volte su frequenze diverse** per robustezza, il che riduce ulteriormente il rate effettivo. Per applicazioni che richiedono downlink frequente o payload più grandi, Sigfox non è adatta.

NB-IoT (Narrowband IoT)

NB-IoT è la risposta degli operatori telefonici alla sfida LPWAN: uno standard **cellulare** (3GPP, introdotto in LTE Release 13) che usa lo **spettro licenziato** già in possesso degli operatori. A differenza di LoRa e Sigfox, funziona sulla stessa infrastruttura delle reti 4G LTE con tre modalità di deployment: **in-band** (usa risorse LTE esistenti), **guard-band** (sfrutta le bande di guardia LTE inutilizzate), **standalone** (banda dedicata, tipicamente GSM riconvertita).

Il vantaggio principale è la **qualità di servizio garantita**: lo spettro licenziato non è condiviso con altri, la copertura esistente degli operatori è capillare, e la latenza è più prevedibile (1.6–10s). Supporta fino a 50.000 dispositivi per cella e ha data rate 250 kbps — superiore a LoRa. Richiede SIM card e abbonamento all'operatore.

Confronto LPWAN

Feature	LoRaWAN	Sigfox	NB-IoT
Spettro	Unlicensed (ISM)	Unlicensed (ISM)	Licensed (LTE)
Modulazione	CSS	UNB	OFDMA/SC-FDMA
Range	5–15 km	10–50 km	15 km
Max Data Rate	50 kbps	100 bps UL	250 kbps
Payload	Fino a 243 byte	12 byte UL, 8 DL	1600 byte
Msg/giorno	Illimitati (duty cycle)	140 UL, 4 DL	Illimitati
Bidirezionale	Sì (classi A/B/C)	Limitato	Sì
Rete	Privata o pubblica	Sigfox operated	Operatore telco
Sicurezza	AES 128-bit	Proprietaria	LTE-grade

Protocolli di Comunicazione

MQTT (Message Queuing Telemetry Transport)

MQTT è il protocollo di messaggistica più diffuso nell'IoT. Nacque negli anni '90 per monitorare oleodotti via satellite (dove la banda era carissima e le connessioni instabili), e questa origine spiega il suo design: **minimo overhead possibile, massima robustezza alle connessioni intermittenti**.

Il meccanismo fondamentale è **publish/subscribe**: invece della comunicazione diretta client-server, ogni messaggio viene inviato a un **topic** (una stringa che identifica l'argomento), e il broker lo consegna a tutti i client che si sono iscritti a quel topic. Publisher e subscriber non si conoscono e non devono essere connessi contemporaneamente. Il protocollo gira su TCP/IP (porta 1883 in chiaro, 8883 con TLS), è standard OASIS e ISO (20922), e ha un header minimo di soli **2 byte** — il più compatto tra i protocolli IoT.

Architettura

Il sistema ha tre attori principali:

- Il **Broker** è il server centrale: riceve tutti i messaggi dai publisher, mantiene le sottoscrizioni, e consegna i messaggi ai subscriber appropriati. È l'unico componente «always-on» del sistema.
- Il **Publisher** è un client che invia messaggi: si connette al broker, pubblica su un topic, e può disconnettersi. Non sa chi lo ascolta.
- Il **Subscriber** è un client che riceve messaggi: si connette al broker, si iscrive a uno o più topic, e riceve tutti i messaggi pubblicati su quei topic mentre è connesso.

Il sistema è **completamente disaccoppiato** in tre dimensioni, che è il punto di forza di MQTT rispetto alla comunicazione diretta:

- **Spazio**: publisher e subscriber non conoscono i reciproci indirizzi IP — comunicano solo attraverso il broker e il topic
- **Tempo**: non devono essere connessi simultaneamente — i messaggi possono essere conservati (Persistent Sessions, QoS 1/2) per la riconnessione
- **Sincronizzazione**: le operazioni sono completamente asincrone — pubblicare un messaggio non blocca il publisher in attesa di una risposta

Topic

I topic sono stringhe gerarchiche che usano `/` come separatore, simili a path di filesystem. Ad esempio `home/livingroom/temperature` identifica la temperatura del salotto di casa. Sono case-sensitive e possono avere qualsiasi profondità. Ogni livello della gerarchia rappresenta un aspetto logico del sistema.

Per i subscriber, MQTT supporta due wildcard che permettono di iscriversi a classi di topic:

- `+` (single-level wildcard): rimpiazza **esattamente un livello** della gerarchia. `home/+/temperature` matcha `home/livingroom/temperature` e `home/bedroom/temperature`, ma non `home/floor1/livingroom/temperature`.
- `#` (multi-level wildcard): rimpiazza **zero o più livelli** dalla posizione in avanti. `home/#` matcha tutto ciò che inizia con `home/` a qualsiasi profondità. Il `#` deve essere sempre l'ultimo carattere del topic filter.

I topic che iniziano con `$` sono riservati ai **system topic** del broker, non pubblicabili da client normali (es. `$SYS/broker/clients/connected` riporta il numero di client connessi al broker).

QoS (Quality of Service)

MQTT offre tre livelli di garanzia sulla consegna dei messaggi, con un tradeoff esplicito tra affidabilità e overhead:

QoS 0 — At most once («fire and forget»): il publisher invia il messaggio, il broker lo consegna ai subscriber, nessun ACK in nessuna direzione. Il messaggio può andare perso se la connessione cade durante la trasmissione. Adatto per dati che si aggiornano frequentemente (temperatura ogni secondo) dove la perdita occasionale è accettabile.

QoS 1 — At least once: la consegna è **garantita**, ma possono arrivare duplicati. Il publisher invia PUBLISH e attende un PUBACK dal broker; se il PUBACK non arriva entro il timeout, rimanda il messaggio con il flag DUP impostato. Il subscriber può quindi ricevere lo stesso messaggio più volte — deve essere idempotente. Adatto per dati importanti dove la perdita non è accettabile ma i duplicati sono gestibili.

QoS 2 — Exactly once: consegna garantita e **senza duplicati**, tramite un handshake a 4 step: PUBLISH → PUBREC (broker ha ricevuto) → PUBREL (publisher conferma) → PUBCOMP (broker ha consegnato). Il meccanismo a 4 step garantisce che il messaggio sia consegnato esattamente una volta anche in caso di disconnessioni durante il processo. Il prezzo è l'overhead più alto. Adatto per comandi critici (es. «apri valvola») dove sia la perdita che la doppia esecuzione sono problemi.

Funzionalità chiave

Retained Messages: normalmente un subscriber che si connette dopo una pubblicazione non vede i messaggi passati. Con un messaggio «retained» (flag RETAIN=1), il broker salva l'**ultimo messaggio pubblicato su quel topic** e lo consegna immediatamente a ogni nuovo subscriber. Utilissimo per lo stato corrente di un dispositivo: un sensore pubblica ogni 5 minuti con RETAIN=1, e chiunque si iscriva vede subito l'ultimo valore senza aspettare 5 minuti.

Last Will and Testament (LWT): quando un client IoT perde connessione improvvisamente (batteria scarica, crash, interruzione di rete), il broker non riceve un normale pacchetto di disconnessione. L'LWT è un messaggio che il client specifica al momento della connessione (nel pacchetto CONNECT) e che il broker pubblica automaticamente su un topic specifico se il client si disconnette in modo inatteso. Permette agli altri client di rilevare la disconnessione involontaria di un dispositivo.

Persistent Sessions (cleanStart = false): con la sessione persistente, il broker memorizza le sottoscrizioni e i messaggi QoS 1/2 in attesa per un client anche quando è offline. Alla riconnessione, il client riceve automaticamente tutti i messaggi che ha perso. Senza sessione persistente (cleanStart = true), ogni connessione parte da zero.

Keep Alive: TCP non notifica immediatamente la disconnessione silenziosa (es. perdita di segnale). Il meccanismo Keep Alive di MQTT risolve questo: il client invia periodicamente un pacchetto PINGREQ, il broker risponde con PINGRESP. Se il broker non riceve nessuna comunicazione dal client entro $1.5 \times$ il periodo keepAlive configurato, lo dichiara disconnesso e pubblica il LWT.

Formato messaggio

Ogni pacchetto MQTT è composto da tre parti:

- **Fixed Header** (presente in **ogni** tipo di pacchetto, 2+ byte): nei primi 4 bit c'è il Message Type (CONNECT, PUBLISH, SUBSCRIBE, ecc.), nei successivi 4 bit ci sono i flag DUP/QoS/RETAIN, poi il Remaining Length codificato con un algoritmo a lunghezza variabile (1–4 byte, che permette di rappresentare payloads fino a 256 MB con 4 byte).
- **Variable Header**: presente solo in certi tipi di pacchetti, contiene informazioni specifiche al tipo (es. il Packet Identifier nei pacchetti con QoS 1/2, il Protocol Name nel CONNECT).
- **Payload**: il contenuto del messaggio, specifico al tipo di pacchetto (PUBLISH porta il dato applicativo; CONNECT porta clientId, will topic, username, password).

CONNECT packet fields

Il pacchetto CONNECT è il primo che un client invia al broker, e contiene tutta la configurazione della sessione: `clientId` (stringa univoca che identifica il client presso il broker — il broker usa questo ID per le sessioni persistenti), `cleanStart` (true = inizia una sessione pulita, false = riprendi la sessione precedente), i campi LWT (`willTopic`, `willQoS`, `willMessage`, `willRetain`), `keepAlive` (intervallo in secondi), e opionalmente `username` / `password` per l'autenticazione.

Codici di ritorno CONNACK

Il broker risponde al CONNECT con un CONNACK che include un codice di ritorno. Conoscere questi codici è utile per il debugging:

- 0 = Connection accepted
- 1 = Bad protocol version (il broker non supporta la versione MQTT richiesta)
- 2 = Client ID rejected (`clientId` non valido o già in uso)
- 3 = Server unavailable (il broker esiste ma non accetta connessioni al momento)
- 4 = Bad credentials (username/password errati)
- 5 = Not authorized (il client non ha i permessi per connettersi)

SUBACK return codes

Quando un client si iscrive a un topic con SUBSCRIBE, il broker risponde con un SUBACK che include un codice per ogni topic richiesto: 0, 1, o 2 indicano che la sottoscrizione è stata accettata con quel livello di QoS massimo (il broker può abbassare il QoS richiesto), mentre 128 indica che la sottoscrizione è stata rifiutata.

MQTT 5 — novità principali

MQTT 5 (2019) ha introdotto miglioramenti significativi rispetto a MQTT 3.1.1 per supportare scenari enterprise e IoT più complessi:

- **User Properties:** coppie key-value custom aggiungibili a qualsiasi tipo di pacchetto MQTT. Permettono di trasportare metadati applicativi arbitrari (es. timestamp, device type, tenant ID) senza modificare il payload applicativo, utili per routing e filtering a livello broker.
- **Reason Codes estesi:** MQTT 3 usava codici di errore molto limitati. MQTT 5 introduce reason codes dettagliati per **tutti** i tipi di ACK, sia positivi che negativi (es. un PUBACK può indicare «quota exceeded» o «payload format invalid» invece di un generico fallimento), rendendo il debugging molto più efficace.
- **Shared Subscriptions:** permettono il **load balancing lato subscriber**. Più subscriber si iscrivono allo stesso topic usando il prefisso `$share/{nome-gruppo}/topic`. Il broker distribuisce i messaggi in round-robin tra i subscriber del gruppo, invece di inviarli a tutti. Essenziale per scalare il processing dei messaggi orizzontalmente.

CoAP (Constrained Application Protocol)

CoAP è la risposta dell'IETF alla domanda: «come portiamo il modello REST — che funziona così bene sul web — sui dispositivi IoT più vincolati?» HTTP è troppo pesante per microcontrollori con pochi KB di RAM e una batteria da anni. CoAP risolve il problema reimplementando le idee di HTTP (metodi GET/POST/PUT/DELETE, codici di risposta, URI) su un trasporto **UDP** con un header di soli **4 byte fissi**, mantenendo la semantica RESTful ma eliminando tutto l'overhead non necessario. Definito in RFC 7252 (IETF), opera sulla porta 5683.

Perché non HTTP?

La domanda è legittima: HTTP è universale, ben supportato, perché non usarlo direttamente? Il problema è che HTTP+TCP è progettato per connessioni stabili e affidabili tra computer con risorse abbondanti, e porta un overhead insostenibile per i dispositivi IoT vincolati:

- **Header enormi:** un header HTTP in ASCII può essere lungo centinaia di byte (solo il campo Host, Content-Type e Accept possono superare i 100 byte). CoAP ha un header fisso di **4 byte** — due ordini di grandezza meno.
- **TCP è costoso:** stabilire una connessione TCP richiede un 3-way handshake (3 round trip prima di trasmettere), e mantenere la connessione richiede stato (finestre di congestione, buffer di ritrasmissione) che occupa RAM preziosa su un MCU con 2–8 KB di SRAM.
- **Il modello QoS di TCP non mappa bene:** TCP garantisce la consegna a livello di flusso di byte, ma non per singoli messaggi. CoAP invece modella esplicitamente quattro tipi di messaggio con semantica precisa (vedi sotto), dando al programmatore controllo granulare sulla garanzia di consegna.
- **Stato connessione = consumo batteria:** un MCU a batteria deve dormire il più possibile. TCP richiede che la connessione resti aperta (keepalive, ritrasmissioni periodiche). CoAP su UDP è **stateless** — ogni messaggio è indipendente, il dispositivo può dormire tra un messaggio e l'altro senza problemi.

Tipi di messaggio

CoAP definisce quattro tipi di messaggio che, insieme, forniscono la granularità di controllo che TCP non offre a livello di messaggio:

- **CON** (Confirmable): il mittente richiede una conferma esplicita dal destinatario. Se l'ACK non arriva entro il timeout, il messaggio viene ritrasmesso con exponential backoff (vedi sotto). Usato quando la consegna è critica.
- **NON** (Non-confirmable): il mittente invia senza aspettare ACK. Fire-and-forget puro, nessuna garanzia di consegna. Usato per dati periodici dove la perdita occasionale è accettabile (es. temperatura ogni 10 secondi).
- **ACK:** inviato in risposta a un CON per confermare la ricezione. Può essere **piggybacked** (include già la risposta applicativa) o **vuoto** (solo conferma, la risposta arriverà in un secondo momento).
- **RST** (Reset): indica che un CON è stato ricevuto ma non può essere processato (es. il contesto della richiesta non esiste più). Usato per sincronizzazione dello stato.

Metodi e codici risposta

I metodi CoAP mappano direttamente sulle operazioni REST: GET (leggi risorsa), POST (crea o processa risorsa), PUT (aggiorna risorsa), DELETE (elimina risorsa). La semantica è identica a HTTP, il che rende CoAP familiare a chi conosce il web.

I codici di risposta usano il formato X.YY (ispirato a HTTP): 2.01 Created, 2.05 Content (equivalente di 200 OK), 4.04 Not Found, 4.05 Method Not Allowed, 5.00 Internal Server Error. Il primo numero indica la classe (2.xx = successo, 4.xx = errore client, 5.xx = errore server), il secondo il codice specifico.

Header (4 byte fissi)

L'header CoAP è compatto per design: 2 bit di Version (sempre 1 nella versione corrente), 2 bit di Type (indica CON/NON/ACK/RST), 4 bit di Token Length, 8 bit di Code (metodo o codice risposta), 16 bit di Message ID (usato per correlazione e deduplicazione). Dopo i 4 byte fissi ci sono il Token (da 0 a 8 byte, per correlare richiesta e risposta), le Options (in formato delta encoding per compattezza), il byte marker `0xFF` (separatore), e il Payload.

Meccanismi di affidabilità

Packet loss e retransmission: solo i messaggi CON vengono ritrasmessi automaticamente in caso di perdita. Il timer di retransmission usa **exponential backoff**: inizia con un timeout casuale tra 1 e 1.5 secondi, poi lo raddoppia ad ogni tentativo ($1s \rightarrow 2s \rightarrow 4s \rightarrow 8s \rightarrow 16s...$) fino a un massimo di `MAX_RETRANSMIT = 4` tentativi. Dopo 4 tentativi senza ACK, il messaggio viene considerato perso e la connessione «broken». I messaggi NON non vengono mai ritrasmessi — sono fire-and-forget assoluti.

Risposta separata (deferred): a volte il server riceve una richiesta CON ma l'elaborazione richiede tempo (es. leggere un sensore lento o interrogare un database). Se il server aspettasse per rispondere, il client continuerebbe a ritrasmettere la richiesta pensando che sia andata persa. La soluzione è la **risposta separata**: il server invia immediatamente un **ACK vuoto** (solo conferma, senza risposta) per fermare le ritrasmissioni del client, poi quando ha il risultato lo invia come nuovo messaggio CON separato, e il client risponde con un ACK.

Observe (RFC 7641)

Observe è un'estensione di CoAP che aggiunge un comportamento simile al pub/sub mantenendo il modello RESTful. Funziona così: il client invia una GET con l'opzione Observe=0 su una risorsa; il server la registra come «osservatore» di quella risorsa e, da quel momento, invia automaticamente notifiche ogni volta che il valore della risorsa cambia. Per de-registrarsi, il client invia una GET con Observe=1. Il vantaggio rispetto al polling continuo è enorme: invece di interrogare il server ogni secondo, il client riceve aggiornamenti solo quando c'è qualcosa di nuovo.

Resource Discovery

Come fa un client di scoprire quali risorse sono disponibili su un server CoAP senza documentazione a priori? CoAP riserva l'URI `/.well-known/core` per il **discovery**: una GET su questo URI restituisce una lista di tutte le risorse disponibili nel formato **CoRE Link Format** (RFC 6690), che include URI, tipi di risorsa e media type supportati. È l'equivalente CoAP di un file WSDL.

Sicurezza

CoAP usa **DTLS** (Datagram TLS, la versione di TLS per UDP) per cifrare il canale. Quattro modalità progressive:

- **NoSec**: nessuna sicurezza (solo per test o reti completamente isolate)
- **PreSharedKey**: chiave simmetrica pre-condivisa tra device e server
- **RawPublicKey**: coppie di chiavi asimmetriche senza certificati (meno overhead di PKI)
- **Certificate**: PKI completa con certificati X.509 (massima sicurezza, più overhead)

CoAP extensions

Multicast: poiché CoAP gira su UDP, supporta nativamente il **group communication** tramite UDP multicast. Utile per discovery di gruppo (chiedere «chi siete?» a tutti i dispositivi su una rete) o per inviare comandi a gruppi di dispositivi in un solo messaggio.

HTTP↔CoAP proxy: un proxy bidirezionale permette di far comunicare client HTTP con server CoAP e viceversa, traducendo automaticamente tra i due protocolli. Il proxy può anche fare **caching** delle risposte CoAP (usando l'opzione max-age), riducendo il traffico verso i dispositivi vincolati.

Resource Directory: in reti con molti nodi CoAP, un registro centralizzato — la Resource Directory — raccoglie le risorse annunciate da tutti i nodi. Alternativa scalabile a `/.well-known/core` per reti grandi dove non si vuole fare discovery su ogni singolo nodo.

AMQP (Advanced Message Queuing Protocol)

AMQP è un protocollo di messaggistica nato in ambito bancario e finanziario, progettato per ambienti enterprise dove l'affidabilità e l'interoperabilità tra sistemi di vendor diversi sono non negoziabili. A differenza di MQTT (che punta alla semplicità e al minimo overhead), AMQP punta alla **ricchezza funzionale**: routing flessibile, transazioni, ACK espliciti, dead letter queues.

I motivi per scegliere AMQP rispetto ad altri protocolli: è **cross-organization** (funziona tra organizzazioni e tecnologie diverse, indipendentemente dal vendor), **cross-technology** (lo stesso protocollo implementato in Java, Python, Go, .NET), **asincrono** (publisher e subscriber completamente disaccoppiati nel tempo), standard **aperto** (non vendor-locked come molte soluzioni proprietarie), **sicuro** (autenticazione SASL, crittografia TLS), e **affidabile** con transazioni, ACK e dead letter queues per i messaggi non consegnabili.

Il broker AMQP ha un'architettura interna a tre componenti che lavorano insieme: le **Queues** (code che bufferizzano i messaggi per i consumer), gli **Exchanges** (entità che ricevono i messaggi dai publisher e applicano le regole di routing), e i **Bindings** (le regole che connettono un Exchange a una Queue, specificando i criteri di routing).

Il flow è sempre: Publisher → Exchange (applica routing) → Queue → Consumer. La flessibilità di AMQP sta negli Exchange, che possono instradare lo stesso messaggio a zero, una o molte code a seconda del tipo:

- **Direct Exchange**: instrada il messaggio alla Queue il cui binding key corrisponde **esattamente** alla routing key del messaggio. Usato per routing preciso e point-to-point.
- **Fanout Exchange**: ignora completamente la routing key e manda il messaggio a **tutte** le Queue bound all'exchange. Il broadcast classico: un evento pubblicato raggiunge tutti i consumer iscritti.
- **Topic Exchange**: la routing key del messaggio viene confrontata con un pattern usando wildcard (* matcha una singola parola, # matcha zero o più parole). Permette routing gerarchico flessibile simile ai topic MQTT.
- **Headers Exchange**: il routing non usa la routing key ma gli header del messaggio (coppie key-value). Più espressivo ma anche più lento da valutare.

BLE (Bluetooth Low Energy)

GAP (Generic Access Profile)

Gestisce advertising e setup delle connessioni. Ruoli:

- **Central**: scansiona in cerca di peripheral e avvia la connessione (es. smartphone)
- **Peripheral**: fa advertising e accetta connessioni (es. sensore)

Advertising: il peripheral trasmette pacchetti periodicamente (ogni 20ms–10.24s) sui canali 37, 38, 39 (dedicati per evitare interferenze Wi-Fi). Tipi principali:

- `ADV_IND`: connectable, undirected (chiunque può connettersi)
- `ADV_NONCONN_IND`: non-connectable (beacon, solo broadcast)
- `ADV_DIRECT_IND`: directed verso un central specifico

GATT (Generic Attribute Profile)

Struttura dati gerarchica per lo scambio dati una volta connessi:

Profile → Service → Characteristic → Descriptor

- **Service**: raggruppa caratteristiche correlate, identificato da UUID (es. «Heart Rate Service» = 0x180D)

- **Characteristic:** singolo dato con UUID, valore e properties
- **Properties** (operazioni permesse sulla characteristic):
 - `Read`, `Write`, `Write Without Response`
 - `Notify`: il server invia aggiornamenti senza ACK
 - `Indicate`: il server invia aggiornamenti **con** ACK dal client
- **Descriptor:** metadati sulla caratteristica; il più importante è il **CCCD** (Client Characteristic Configuration Descriptor) che il client scrive per abilitare Notify o Indicate

Parametri di connessione e sicurezza

Connection parameters: `connInterval` (7.5ms–4s, frequenza degli slot di comunicazione), `slaveLatency` (quanti interval il peripheral può saltare se non ha dati), `supervisionTimeout` (dopo quanto dichiarare la connessione persa).

Sicurezza — Pairing: Just Works (senza PIN, vulnerabile a MITM), Passkey Entry (PIN a 6 cifre), Out-of-Band (NFC o QR). **Bonding:** dopo il pairing le chiavi vengono salvate per evitare il pairing ad ogni riconnessione.

Zigbee

Protocollo mesh basato su **IEEE 802.15.4** (PHY + MAC). Zigbee aggiunge i layer **Network (NWK)** e **Application (APL)** sopra.

Device types

- **Coordinator** (FFD): uno per rete; avvia la PAN, assegna indirizzi, mantiene la routing table, può essere Trust Center
- **Router** (FFD): instrada traffico, può avere figli, estende la copertura della rete
- **End Device** (RFD): foglia della rete; può dormire (battery-friendly); comunica solo con il proprio parent (coordinator o router)

Topologie

- **Star:** tutti i device parlano solo col coordinator
- **Tree:** coordinator al vertice, router come rami, end device come foglie
- **Mesh:** i router si connettono tra loro con percorsi multipli → resiliente ai guasti (auto-healing)

Profili applicativi e sicurezza

Profili: Home Automation (HA), Smart Energy (SE), ZigBee Light Link (ZLL), Building Automation (BA) — ogni profilo definisce cluster e attributi standard.

Sicurezza: cifratura **AES-128**. Il **Trust Center** (solitamente il coordinator) distribuisce le chiavi: **network key** (condivisa da tutta la rete) e **link key** (chiave punto-a-punto tra due device).

Confronto protocolli

Feature	MQTT	CoAP	HTTP	AMQP
Trasporto	TCP	UDP	TCP	TCP
Modello	Pub/Sub	Req/Resp	Req/Resp	Pub/Sub + Queue
Header	2 byte min	4 byte fissi	Grande (testo)	8 byte min

Feature	MQTT	CoAP	HTTP	AMQP
Overhead	Molto basso	Molto basso	Alto	Medio
Affidabilità	QoS 0/1/2	CON/NON	TCP	Transazioni
Consumo	Basso	Molto basso	Alto	Medio
Encoding	Binario	Binario	Testo (ASCII)	Binario
Sicurezza	TLS/SSL	DTLS	TLS/SSL	TLS/SASL
Porta	1883/8883	5683/5684	80/443	5672

Web of Things (WoT)

Standard **W3C** per rendere i dispositivi IoT accessibili via web con interfacce standardizzate e indipendenti dal protocollo fisico sottostante.

Problema che risolve

Ogni vendor ha API proprietarie → impossibile interoperabilità tra ecosistemi diversi. WoT definisce un **livello semantico comune** sopra i protocolli esistenti (HTTP, MQTT, CoAP, WebSocket) senza rimpiazzarli.

Thing Description (TD)

Il componente centrale di WoT. È un documento **JSON-LD** che descrive un dispositivo: le sue proprietà, azioni ed eventi, i metadati semantici, e i binding di protocollo. È la «business card» della cosa — chiunque la legga sa come interagire con il device.

Interaction Affordances

I tre tipi di interazione che una Thing può esporre:

- **Properties:** stato leggibile e/o scrivibile (es. temperatura corrente, stato luce on/off)
- **Actions:** operazioni invocabili che causano cambiamenti (es. accendi luce, avvia motore)
- **Events:** notifiche asincrone emesse dalla Thing (es. allarme fumo, soglia superata)

Protocol Bindings

La stessa TD può essere implementata su protocolli diversi. I **Binding Templates** specificano come mappare le Interaction Affordances su messaggi reali:

- Una Property «temperature» può essere letta via HTTP GET, oppure via MQTT subscribe, oppure via CoAP GET
- Il binding descrive URL/topic, metodo, content type, forma dei dati

Semantic Interoperability

Uso di **JSON-LD** + ontologie standard (schema.org, iotschema.org) per dare **significato** ai dati, non solo struttura. Un «temperature» con `@type: "Temperature"` è comprensibile da qualsiasi sistema che conosce l'ontologia, indipendentemente dall'implementazione.

WoT Building Blocks

- **Thing Description:** descrive la cosa (struttura, interfacce, metadati)
- **Binding Templates:** come tradurre la TD in messaggi concreti sul protocollo scelto
- **Scripting API:** come scrivere applicazioni WoT che consumano o espongono Things

Formati di serializzazione dati

JSON: leggibile, key-value, ampiamente usato nelle API web. Payload più grande (text-based).

CBOR (Concise Binary Object Representation): formato binario basato su JSON, più compatto. RFC 7049. Auto-descrittivo.

Protocol Buffers (Protobuf): serializzazione binaria di Google. Richiede schema `.proto`. Molto compatto e veloce, ma non auto-descrittivo.

Task Offloading

Sensor Fusion

Ogni sensore ha una visione **parziale e rumorosa** della realtà: un GPS funziona all'aperto ma è cieco al chiuso; un accelerometro è preciso sul breve termine ma accumula deriva nel tempo; un sensore di temperatura legge la temperatura in un singolo punto, non quella media della stanza. La **sensor fusion** combina i dati di più sensori per compensare le debolezze di ognuno e ottenere una rappresentazione più accurata, completa e affidabile della realtà.

L'intuizione è semplice: GPS + accelerometro per la localizzazione indoor/outdoor combinano il meglio dei due — il GPS corregge la deriva dell'accelerometro all'aperto, l'accelerometro tiene traccia della posizione quando il GPS non ha segnale. Il risultato è migliore di entrambi da soli.

Pre-processing per la fusion

Prima di fondere i dati da sensori diversi, occorre risolvere tre problemi pratici che emergono quasi sempre in sistemi reali:

1. **Allineamento temporale**: sensori diversi campionano a frequenze diverse (es. accelerometro a 100 Hz, GPS a 1 Hz). Prima di fonderli, i valori devono essere interpolati o ricampionati su una scala temporale comune.
2. **Allineamento spaziale**: sensori fisicamente posizionati in punti diversi (es. un accelerometro sul polso e uno sul torace) misurano la stessa grandezza ma da prospettive diverse. Servono trasformazioni di coordinate per portarli allo stesso sistema di riferimento.
3. **Normalizzazione**: grandezze diverse hanno scale e unità diverse (gradi Celsius, accelerazione in m/s^2 , distanza in cm). Prima di combinarli, i valori vanno normalizzati in un range comune.

Architetture

- **Centralizzata**: tutti i sensori inviano i dati grezzi a un nodo centrale che esegue la fusion. Vantaggio: il nodo centrale ha la massima informazione disponibile per fare la migliore fusion possibile. Svantaggio: alto traffico di rete, single point of failure, non scala bene con molti nodi.
- **Distribuita**: ogni nodo fonde localmente i propri sensori e condivide con gli altri solo il risultato parziale (feature estratte, non dati grezzi). Vantaggio: efficiente in termini di banda, resiliente ai guasti. Svantaggio: le correlazioni tra sensori su nodi diversi vengono perse.
- **Ibrida**: fusion locale di nodo + aggregazione centrale dei risultati parziali. È il modello più comune nei sistemi reali: riduce il traffico ma mantiene una visione globale sufficiente per le decisioni finali.

Framework Concettuale dell'Offloading

Il **task offloading** è la decisione di dove eseguire una computazione: localmente sul dispositivo IoT, su un nodo edge nelle vicinanze, o nel cloud. Non è una scelta banale — dipende da energia disponibile, potenza computazionale, latenza della rete, e dimensione dei dati da trasmettere.

Il framework concettuale per prendere questa decisione risponde a quattro domande in sequenza: **IF** (conviene offloadare, o è meglio fare tutto in locale?), **WHAT** (quale parte del task offloadare?), **WHEN**

(in quale momento, considerando le condizioni della rete?), **WHERE** (verso quale nodo — edge vicino, edge lontano, cloud?).

Le 4 possibilità computazionali

- **Local**: il device IoT esegue interamente il task in locale. Zero latenza di rete, massima privacy, ma limitato dalla capacità computazionale e dall'energia disponibile. Ottimale per task semplici e frequenti (es. campionamento e threshold checking).
- **Edge (full offloading)**: il task viene spostato interamente al nodo edge (MEC). Il device risparmia energia di elaborazione, la latenza è bassa grazie alla prossimità geografica. Ottimale quando il task è computazionalmente pesante ma i dati da trasmettere sono pochi.
- **Split Computing**: il task è diviso — i primi layer (es. i layer di una rete neurale) girano sul device, i layer successivi sull'edge. La divisione può essere **statica** (fissa a design time, semplice) o **dinamica** (adattiva a runtime in base alle condizioni della rete). Il layer ottimale di split è $L^* = \arg \min_L E(L)$ — quello che minimizza l'energia totale del sistema.
- **Fluid Computing**: estensione del split computing dove il task è spezzato in **funzioni** che vengono assegnate a dispositivi diversi in pipeline (device → edge → cloud → edge → device). L'algoritmo di assegnazione greedy assegna ogni funzione al dispositivo che minimizza il contributo energetico marginale considerando sia il calcolo che la trasmissione.

Condizione energetica fondamentale

La decisione di offloadare dipende dal confronto tra due costi energetici:

$E_d < E_e + E_n \rightarrow$ conviene eseguire localmente; $E_d > E_e + E_n \rightarrow$ conviene fare offloading.

Dove E_d è l'energia per eseguire il task localmente, E_e è l'energia che il server edge consuma per elaborarlo (proporzionalmente al device, pesata per efficienza), e E_n è l'energia per trasmettere i dati via rete. In pratica, E_n (la trasmissione WiFi) è spesso il termine dominante: trasmettere dati costa più che elaborarli localmente.

Un **esempio reale con ESP32** lo conferma: eseguire un task on-device costa **0.02 mAh**, fare full offloading via WiFi costa **0.052 mAh** — il locale è $2.6\times$ più efficiente. Questo spiega perché TinyML e Split Computing siano così importanti: ridurre la trasmissione è la chiave per l'efficienza energetica.

Offloading: Transform (Trasformazione del Segnale)

Prima di trasmettere dati per l'offloading, conviene spesso **ridurne la dimensione** applicando una trasformazione in locale. L'intuizione è che il segnale grezzo contiene molta ridondanza che può essere eliminata senza perdere l'informazione utile per il task a valle. Meno dati da trasmettere = meno energia radio = maggiore autonomia.

Tecniche su serie temporali

PAA (Piecewise Aggregate Approximation): la serie temporale viene divisa in w segmenti di uguale lunghezza, e ogni segmento viene sostituito con la sua **media**. Si riducono n campioni a w valori ($w \ll n$), ottenendo una versione «bassa risoluzione» della serie. Esempio: 1000 campioni di temperatura in 5 minuti → 20 medie (una ogni 15 secondi). Il pattern generale (trend, salti, ciclicità) è preservato; i dettagli ad alta frequenza vengono perduti.

SAX (Symbolic Aggregate approXimation): estende PAA aggiungendo **quantizzazione simbolica**. Dopo aver applicato PAA e normalizzato la serie, ogni valore PAA viene mappato in una **lettera dell'alfabeto** (es. “a”, “b”, “c”, “d”) usando soglie scelte per rendere ogni simbolo equiprobabile sulla distribuzione gaussiana. La serie temporale diventa così una **stringa di caratteri** — compressione estrema che permette di usare algoritmi di string matching per trovare pattern simili.

Tecniche su immagini

Quando il dato da trasmettere è un'immagine (es. da una camera su un dispositivo edge per object detection), le tecniche di riduzione dimensionale diventano critiche:

- **Riduzione risoluzione:** trasmettere a 480p invece di 1080p riduce la dimensione di circa 5×
- **Scala di grigi:** molte task di object detection non richiedono colori — eliminare i 2 canali colore su 3 riduce i dati di un fattore 3×
- **Compressione JPEG/WebP:** con qualità ridotta (es. 60%) si può ottenere un altro fattore 5–10× con perdita visiva accettabile

Sfide

- **Perdita di informazione:** tutte queste trasformazioni sono lossy. Per alcune task (es. rilevamento di spike anomali su una serie PAA) la perdita di dettaglio può compromettere la qualità del risultato.
- **Dipendenza dal dato:** PAA funziona bene per serie smooth con variazioni lente, ma perde i picchi brevi che potrebbero essere anomalie importanti.
- **Overhead locale:** il tempo e l'energia per eseguire la trasformazione sul device devono essere inclusi nel bilancio energetico totale — non è gratis.
- **Accordo sull'encoding:** device e edge devono concordare a priori lo stesso schema di trasformazione (dimensione dei segmenti PAA, alfabeto SAX). Cambiare il modello richiede aggiornamento di entrambi i lati.

Mobile Edge Computing (MEC)

I dispositivi mobili hanno tre vincoli fondamentali: batteria limitata, CPU lenta, memoria scarsa. Le applicazioni IoT più sofisticate — riconoscimento oggetti in tempo reale, AR/VR, NLP on-device — richiedono risorse che i dispositivi mobili non hanno. Il cloud avrebbe le risorse, ma il round-trip WAN (tipicamente 50–100ms, a volte centinaia) introduce una latenza inaccettabile per applicazioni real-time.

MEC (Mobile Edge Computing) risolve il problema posizionando risorse di calcolo **fisicamente vicine ai dispositivi mobili**: server co-locati con le base station delle reti cellulari o con gli access point WiFi, a distanza di uno o pochi hop. La latenza scende a 1–10ms (un ordine di grandezza migliore del cloud) mentre le risorse computazionali sono molto superiori a quelle del device.

L'architettura ETSI MEC comprende: il **UE** (dispositivo utente, il client che fa offloading), il **MEC Host** (il server fisicamente vicino, co-locato con la base station), la **MEC Platform** (il sistema operativo dell'host che gestisce le applicazioni MEC, il DNS locale, il service registry), e il **MEC Orchestrator** (il livello di gestione che coordina il lifecycle delle applicazioni MEC su più host, decidendo il placement ottimale).

Formulazione matematica

Per ottimizzare le decisioni di offloading, è necessario un modello matematico che quantifichi i costi. Un task viene descritto dalla tupla $(\mathbf{B}, \mathbf{C}, \mathbf{T_max})$: B è la dimensione dell'input in bit (quanti dati devo trasmettere), C è il workload computazionale in cicli CPU (quante operazioni devo eseguire), T_max è il delay massimo tollerabile dall'applicazione.

Calcolo locale

Se il task viene eseguito localmente:

- Il **delay** è semplicemente il numero di cicli diviso la frequenza del processore: $T_{\text{local}} = \frac{C}{f_{\text{local}}}$
- L'**energia** dipende dal cubo della frequenza (consumo dinamico del CMOS): $E_{\text{local}} = \kappa \cdot C \cdot f_{\text{local}}^2$ dove $\kappa \approx 10^{-28}$ è il coefficiente di capacitanza effettiva del chip. Si noti che abbassare la frequenza riduce l'energia quadraticamente — è il principio di **dynamic voltage and frequency scaling** (DVFS).

Offloading

Se il task viene offloadato, il delay e l'energia si articolano su più componenti:

- Il **tasso di trasmissione** è dato dalla formula di Shannon: $r = W \cdot \log_2 \left(1 + \frac{p \cdot h^2}{N_0 \cdot W} \right)$ dove W è la bandwidth, p la potenza trasmessa, h il guadagno del canale, N_0 la densità spettrale del rumore.
- Il **delay totale** è la somma del tempo di trasmissione + tempo di elaborazione sul server + tempo di ritorno: $T_{\text{offload}} = T_{\text{tx}} + T_{\text{server}} + T_{\text{rx}}$ con $T_{\text{tx}} = \frac{B}{r}$ e $T_{\text{server}} = \frac{C}{f_{\text{server}}}$.
- L'**energia di trasmissione** è potenza $\times$ tempo: $E_{\text{tx}} = p \cdot T_{\text{tx}}$.

Obiettivi di ottimizzazione

Il problema di offloading è tipicamente formulato come un'ottimizzazione multi-obiettivo. Il modo classico di gestire due obiettivi (latenza e energia) è combinarli con un parametro di peso:

$$\min J = \alpha \cdot T + (1 - \alpha) \cdot E \text{ con } \alpha \in [0, 1]$$

Con $\alpha = 1$ si minimizza solo la latenza (es. per applicazioni real-time critiche); con $\alpha = 0$ si minimizza solo l'energia (es. per dispositivi a batteria); un valore intermedio bilancia i due obiettivi. I vincoli tipici sono: $T \leq T_{\text{max}}$ (la latenza non può superare il massimo tollerabile), $E \leq E_{\text{max}}$ (il budget energetico), più vincoli sulla banda disponibile e sulla CPU del server.

Multi-User MEC

Nella realtà, il server MEC è condiviso tra molti dispositivi che trasmettono contemporaneamente. Questo introduce il problema dell'accesso multiplo al canale radio. Due approcci principali:

- **OFDMA (Orthogonal Frequency Division Multiple Access)**: lo spettro viene suddiviso in sotto-canali ortogonali e ogni utente ottiene uno o più sotto-canali dedicati. Nessuna interferenza tra utenti, ma la banda disponibile per ognuno è limitata.
- **NOMA (Non-Orthogonal Multiple Access)**: tutti gli utenti trasmettono sullo stesso canale contemporaneamente, a potenze diverse. Il ricevitore usa **SIC (Successive Interference Cancellation)** per decodificare prima il segnale più forte e sottrarlo, poi il secondo, e così via. Più efficiente spettralmente di OFDMA, ma la cancellazione delle interferenze introduce complessità computazionale e potenziale errore di propagazione.

L'ottimizzazione congiunta delle decisioni di offloading (binarie: offloadato o no?) e dell'allocazione delle risorse (continua: quanta banda a chi?) è un problema **MINLP (Mixed-Integer Non-Linear Programming)**, NP-hard in generale.

Approcci risolutivi

Poiché il MINLP esatto non è scalabile, si usano quattro approcci pratici:

- **Rilassamento convesso:** si «rilassano» le variabili binarie (0/1) in variabili continue in $[0,1]$, si risolve il problema convesso risultante in tempo polinomiale, poi si arrotonda il risultato alla soluzione intera più vicina. Semplice, ma l'arrotondamento può degradare la qualità della soluzione.
- **Coordinate Descent:** si alterna tra due sottoproblemi — fissando le decisioni di offloading e ottimizzando l'allocazione delle risorse (problema convesso), poi fissando le risorse e ottimizzando l'offloading. Si itera fino a convergenza. Converge a un ottimo locale, non garantisce l'ottimo globale.
- **Lyapunov Optimization:** framework per ottimizzazione online di sistemi con canali e task che variano nel tempo. Si definiscono code virtuali, e l'obiettivo è minimizzare il **drift-plus-penalty** (la somma del tasso di crescita delle code e del costo di sistema). Garantisce stabilità delle code e convergenza all'ottimo in media senza conoscere la distribuzione dei dati futuri.
- **Deep Reinforcement Learning:** si addestra un agente (DQN o policy gradient) che impara la politica di offloading ottimale dall'esperienza. Gestisce naturalmente stati ad alta dimensione, non richiede un modello del sistema, ma richiede tempo di training e può avere comportamenti imprevedibili durante l'esplorazione.

ARIMA e Reinforcement Learning

Serie temporali

Una **serie temporale** è una sequenza di osservazioni indicizzate nel tempo: $\{X_t : t = 1, 2, \dots\}$. In IoT, quasi ogni dato di un sensore è una serie temporale — temperatura ogni minuto, passi ogni secondo, consumo elettrico ogni ora.

Ogni serie temporale può essere decomposta in quattro componenti: il **Trend** è la direzione di lungo periodo (la temperatura media che sale negli anni), la **Stagionalità** sono le fluttuazioni periodiche e regolari (il traffico che sale ogni mattina alle 8), la **Ciclicità** sono oscillazioni a lungo termine senza periodo fisso (cicli economici), e il **Residuo** è il rumore casuale non spiegabile.

Stazionarietà

Molti modelli statistici richiedono che la serie sia **stazionaria in senso debole**, ovvero che abbia tre proprietà stabili nel tempo: media costante (non cresce né decresce), varianza costante (non diventa più «rumorosa» nel tempo), e autocovarianza che dipende solo dall'intervallo di tempo tra due punti (il lag k) e non dal momento assoluto t .

Una serie con trend o stagionalità marcata non è stazionaria. Prima di modellarla, occorre rimuovere queste componenti. Il modo standard è la **differenziazione**: sostituire ogni valore X_t con la differenza $\Delta X_t = X_t - X_{t-1}$. Una differenziazione rimuove il trend lineare; applicandola due volte si rimuove un trend quadratico.

Per verificare la stazionarietà si usa il **Test ADF (Augmented Dickey-Fuller)**: l'ipotesi nulla H_0 è che la serie abbia una «unit root» (ovvero sia non stazionaria). Se il p-value è < 0.05 , si rifiuta H_0 e la serie è stazionaria. Se invece non si può rifiutare H_0 , occorre differenziare e ritestare.

ACF e PACF

Per capire quale modello ARIMA usare (cioè determinare i valori di p , d , q), si analizzano due funzioni di correlazione:

ACF (Autocorrelation Function) misura la correlazione lineare tra X_t e X_{t+k} per diversi lag k : $\rho(k) = \frac{\text{Cov}(X_t, X_{t+k})}{\text{Var}(X_t)}$. Mostra quanto il valore corrente sia correlato con quello di k passi fa, senza «filtrare» le correlazioni intermedie.

PACF (Partial ACF) misura la correlazione **diretta** tra X_t e X_{t+k} dopo aver rimosso l'effetto dei lag intermedi $k-1, k-2, \dots, 1$. È come chiedere: «quanto contribuisce specificamente il valore di k passi fa, al netto di tutto il resto?»

I pattern di ACF e PACF permettono di identificare il tipo di modello:

- **AR(p)**: l'ACF decade gradualmente verso zero (le correlazioni diminuiscono ma persistono), mentre la PACF si tronca bruscamente dopo lag p (solo i p valori passati contribuiscono direttamente).
- **MA(q)**: l'ACF si tronca bruscamente dopo lag q (solo q errori passati), mentre la PACF decade gradualmente.
- **ARMA(p, q)**: sia ACF che PACF decadono gradualmente — nessuna troncatura netta. In questo caso servono criteri come AIC/BIC per scegliere p e q .

Modelli AR, MA, ARIMA

I modelli ARIMA sono la famiglia di modelli statistici più usata per previsione di serie temporali.

AR(p) — AutoRegressive: il valore corrente è una combinazione lineare dei p valori passati più un termine di errore: $X_t = c + \varphi_1 X_{t-1} + \dots + \varphi_p X_{t-p} + \varepsilon_t$. L'intuizione è semplice: il futuro dipende dal passato recente. Un AR(1) di temperatura dice «la temperatura domani è proporzionale alla temperatura di oggi, più un po' di rumore».

MA(q) — Moving Average: il valore corrente dipende dalla media dei q errori di previsione passati: $X_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}$. Un MA non usa i valori passati direttamente ma gli «shock» passati (scostamenti dalla media). È **sempre stazionario**, a prescindere dai parametri.

ARIMA(p, d, q): il modello generale che combina tutto. p è l'ordine AR (quanti valori passati), d è il grado di differenziazione necessario per ottenere stazionarietà, q è l'ordine MA (quanti errori passati). Il processo è: differenziare la serie d volte per renderla stazionaria, poi adattare un ARMA(p,q) alla serie differenziata.

SARIMA

SARIMA(p,d,q)(P,D,Q)_s estende ARIMA aggiungendo componenti stagionali. I parametri minuscoli riguardano la componente non stagionale (come in ARIMA), i maiuscoli la componente stagionale, e s è il periodo della stagionalità (es. $s = 12$ per dati mensili con stagionalità annuale, $s = 24$ per dati orari con stagionalità giornaliera). La differenziazione stagionale $\Delta_s X_t = X_t - X_{t-s}$ rimuove la componente stagionale.

Box-Jenkins Methodology

Box e Jenkins hanno formalizzato un processo sistematico in 4 fasi per costruire un modello ARIMA:

1. **Identificazione:** tracciare la serie, verificare la stazionarietà con ADF, applicare differenziazione (determinare d), poi analizzare ACF e PACF per identificare p e q candidati.
2. **Stima:** stimare i parametri del modello (φ_i, θ_j) tramite MLE (Maximum Likelihood Estimation) o minimi quadrati.
3. **Diagnostica:** verificare che i **residui del modello siano white noise** (rumore bianco non autocorrelato). Si usa il test di Ljung-Box: se i residui sono autocorrelati, il modello non ha catturato tutta la struttura della serie.
4. **Forecasting:** usare il modello validato per previsioni a h passi in avanti, con intervalli di confidenza.

Per confrontare modelli con diversi (p, q) , si usano criteri di informazione: **AIC** = $2k - 2 \ln(L)$ e **BIC** = $k \cdot \ln(n) - 2 \ln(L)$, dove k è il numero di parametri e L è la likelihood. Valori più bassi indicano modello migliore. BIC penalizza più severamente la complessità.

Anomaly Detection

Rilevare anomalie nelle serie temporali IoT è un problema critico: un picco anomalo di consumo elettrico può indicare un malfunzionamento, un pattern insolito di movimento può indicare una caduta di un paziente. I tre problemi principali sulle serie temporali sono: value forecasting (ARIMA), anomaly detection, e Reinforcement Learning.

Rolling Standard Deviation

Il metodo più intuitivo per anomaly detection è basare le soglie su media e deviazione standard calcolate su una **finestra mobile** di k campioni recenti. Le soglie si adattano dinamicamente al comportamento recente della serie:

$$\text{upper}_t = \mu_t + c \cdot \sigma_t, \quad \text{lower}_t = \mu_t - c \cdot \sigma_t$$

dove μ_t e σ_t sono media e deviazione standard degli ultimi k campioni. Un punto x_t è anomalo se cade fuori dall'intervallo ($c = 2$ è la scelta standard, corrispondente a 5% di falsi positivi per una distribuzione gaussiana; $c = 3$ dà 0.3%).

Il vantaggio principale è la **semplicità e l'adattività**: le soglie seguono automaticamente il trend della serie senza richiedere training. Se la temperatura media sale, le soglie si alzano di conseguenza.

Limiti del Rolling Std — tre problemi strutturali:

- **Auto-normalizzazione degli outlier**: se un punto anomalo entra nella finestra di calcolo, distorce μ e σ verso l'anomalia, abbassando la sensibilità proprio quando servirebbe. L'anomalia si «assorbe» nella stima.
- **Lenta reazione ai regime shift**: con finestre grandi (necessarie per stime stabili), le soglie reagiscono lentamente ai cambi strutturali della serie — falsi positivi durante la transizione.
- **Nessuna memoria contestuale**: il metodo non distingue tra un valore numerico anomalo e un valore «normale» in un momento sbagliato (anomalia contestuale). Un consumo di 1000W alle 3 di notte è anomalo anche se 1000W non supera mai le soglie durante il giorno.

One-Class SVM

La One-Class SVM affronta l'anomaly detection da una prospettiva di machine learning, con un vantaggio fondamentale rispetto ai metodi basati su soglie: **non richiede esempi di anomalie etichettati** — difficilissimi da raccogliere in sistemi IoT reali dove le anomalie sono per definizione rare e imprevedibili.

Si addestra il modello **solo su dati normali**: la One-Class SVM impara la regione dello spazio delle feature che descrive il comportamento normale, tracciando un iperpiano (o ipersferoide con kernel) che racchiude la maggior parte dei dati di training. In inferenza, qualsiasi punto che cade **fuori** da questa regione viene classificato come anomalia.

Il **parametro chiave** $\nu \in (0, 1]$ controlla la tolleranza: rappresenta la percentuale massima di punti di training che il modello può classificare come anomalie (rumore nei dati normali). Con $\nu = 0.05$, il 5% dei dati di training può essere considerato anomalo senza penalizzare il modello.

Vantaggi rispetto al rolling std: non risente degli outlier nel training (il parametro ν gestisce il rumore), può usare kernel non lineari per modellare pattern complessi (es. kernel RBF per comportamenti non gaussiani), e non richiede dati anomali etichettati.

Reinforcement Learning

Il Reinforcement Learning (RL) è il paradigma di machine learning più adatto per problemi di **controllo e decisione sequenziale**: un agente interagisce con un ambiente, riceve feedback (reward) per le sue azioni, e impara nel tempo a massimizzare la ricompensa cumulativa. In IoT è usato per resource allocation, scheduling, offloading, energy management.

Elementi fondamentali

Il framework RL è definito da sei componenti:

- **Agent:** il soggetto che apprende e prende decisioni (es. il controller del sistema IoT)
- **Environment:** tutto ciò con cui l'agente interagisce (la rete IoT, i dispositivi, il canale radio)
- **State** ($s \in S$): la rappresentazione della situazione corrente, tutto ciò che l'agente osserva per decidere
- **Action** ($a \in A$): le possibili scelte dell'agente in ogni stato
- **Reward** (r): il feedback scalare che l'ambiente fornisce dopo ogni azione — positivo per comportamenti desiderabili, negativo per quelli indesiderabili
- **Policy** (π): la strategia dell'agente — un mapping da stati ad azioni che specifica cosa fare in ogni situazione
- **Discount factor** ($\gamma \in [0, 1]$): quanto pesare le reward future rispetto a quelle immediate. $\gamma = 0$ rende l'agente miope (solo reward immediata); $\gamma = 1$ li pesa ugualmente (ma può non convergere se gli episodi sono infiniti).

L'obiettivo è massimizzare il **Return** (ricompensa cumulativa scontata): $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$

MDP (Markov Decision Process)

Il framework matematico che formalizza il problema RL è il **MDP**, definito dalla tupla (S, A, P, R, γ) dove $P(s'|s, a)$ è la probabilità di transizione e $R(s, a)$ è la reward. La **proprietà di Markov** è l'assunzione fondamentale: il futuro dipende solo dallo stato corrente, non dall'intera storia passata. Questa assunzione è il motivo per cui lo stato deve catturare tutta l'informazione rilevante — se non lo fa, le transizioni non sono più markoviane e gli algoritmi standard perdono le garanzie di convergenza.

Value functions e Bellman

Per decidere quale azione è migliore, l'agente usa le **value functions**:

State-Value $V^\pi(s) = E_\pi[G_t | S_t = s]$: valore atteso del return partendo dallo stato s seguendo la policy π . Risponde alla domanda «quanto è buono essere in questo stato?»

Action-Value (Q-function) $Q^\pi(s, a) = E_\pi[G_t | S_t = s, A_t = a]$: valore atteso del return prendendo l'azione a nello stato s e poi seguendo π . Risponde alla domanda «quanto è buona questa azione specifica in questo stato?»

L'**equazione di ottimalità di Bellman** esprime la relazione ricorsiva tra il valore ottimale di uno stato e i valori degli stati successivi: $V^*(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s'|s, a) \cdot V^*(s')]$ In parole: il valore ottimale di uno stato è la migliore azione disponibile, considerando la reward immediata più il valore scontato dello stato futuro.

Q-Learning

Q-Learning è l'algoritmo RL più noto: model-free (non richiede conoscere P e R), off-policy (impara la policy ottimale anche esplorando altre azioni), e basato su temporal difference (aggiorna le stime con ogni singola transizione).

La regola di aggiornamento porta gradualmente Q verso il suo valore ottimale:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Il termine $[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ è il **TD error**: la differenza tra il valore stimato e il valore «target» osservato. α è il learning rate (quanto velocemente aggiornare la stima). L'esplorazione usa la strategia **ϵ -greedy**: con probabilità ϵ si sceglie un'azione casuale (esplorazione), con probabilità $1 - \epsilon$ si sceglie l'azione con Q massimo (sfruttamento). ϵ decresce nel tempo man mano che l'agente impara.

DQN (Deep Q-Network): quando lo spazio degli stati è troppo grande per una tabella Q (es. input è un'immagine o un vettore ad alta dimensione), si usa una rete neurale $Q(s, a; \theta)$ che approssima la funzione Q . Due innovazioni critiche per la stabilità del training: **Experience Replay** (le transizioni

(s, a, r, s') vengono salvate in un buffer e campionate casualmente per il training, rompendo le correlazioni temporali) e **Target Network** (una seconda rete con pesi aggiornati più lentamente usata per calcolare i target di training, evitando l'instabilità da target che si spostano continuamente).

Policy Gradient e Actor-Critic

Un approccio alternativo al Q-Learning è ottimizzare direttamente la policy.

Policy Gradient: si parametrizza la policy $\pi(a|s, \theta)$ come una rete neurale che produce una distribuzione di probabilità sulle azioni, e si aggiornano i parametri θ nella direzione che aumenta il return atteso ($\nabla_{\theta} J(\theta)$). Funziona naturalmente per spazi di azione continui.

Actor-Critic: combina i due approcci. L'**Actor** è la policy (come nel policy gradient), il **Critic** è la value function (come nel Q-learning). Il Critic valuta le azioni dell'Actor e fornisce una stima del vantaggio $A(s, a) = Q(s, a) - V(s)$ (quanto è meglio questa azione rispetto alla media). Usare l'advantage invece del reward grezzo riduce significativamente la varianza del gradiente, rendendo il training più stabile.

Applicazioni RL nell'IoT

RL è stato applicato con successo a: resource allocation (allocare banda e CPU a nodi che cambiano nel tempo), task scheduling (decidere quando e dove eseguire i task), task offloading (la politica di offloading ottimale vista prima), energy harvesting (gestire la carica di batterie con energia raccolta dall'ambiente), network routing (trovare i percorsi ottimali in reti dinamiche), cache placement (decidere cosa memorizzare in cache per ridurre la latenza).

Esempio: Temperature Control (con Q-Learning)

Setup: Stati $S = \{\text{cold, comfortable, hot}\}$. Azioni $A = \{\text{AC, Heating, Nothing}\}$. Reward: +10 se comfortable, -10 se cold/hot (stato indesiderato), -1 per ogni azione (costo energetico — anche fare Nothing ha un costo implicito nello stato sbagliato).

Regola Q-learning con $\alpha = \gamma = 0.5$:

$$Q(s, a) \leftarrow Q(s, a) + 0.5 \left[r + 0.5 \max_{a'} Q(s', a') - Q(s, a) \right]$$

Esempio di un passo: stato corrente = cold, azione scelta = Heating $\rightarrow$ stato successivo $s' = \text{comfortable}$, reward $r = -1 + 10 = +9$ (costo azione + reward stato). Aggiornamento:

$$Q(\text{cold, Heating}) \leftarrow 0 + 0.5 \cdot [9 + 0.5 \cdot 0 - 0] = 4.5$$

Dopo molti episodi e convergenza, la Q-table converge e la policy ottimale emerge: cold $\rightarrow$ Heating, comfortable $\rightarrow$ Nothing, hot $\rightarrow$ AC.

AI on IoT

TinyML

TinyML è il campo che studia come eseguire **inference di machine learning** su microcontrollori e dispositivi ultra-low-power — dispositivi che consumano nell'ordine dei milliwatt o persino microwatt. Il termine «tiny» non è casuale: si tratta di fare ML su hardware con RAM nell'ordine di centinaia di KB e Flash di pochi MB, senza unità in virgola mobile (FPU), spesso con clock di decine o centinaia di MHz.

Perché portare ML direttamente sul dispositivo invece di inviare i dati al cloud? Ci sono sei ragioni fondamentali che si rafforzano a vicenda:

- **Privacy:** i dati grezzi (audio, immagini, dati biometrici) rimangono sul dispositivo — non vengono mai trasmessi. Il dispositivo invia solo il risultato della classificazione.
- **Latenza:** nessun round-trip di rete. La risposta è immediata — critico per applicazioni real-time (es. keyword detection, fall detection).
- **Affidabilità:** funziona anche senza connettività. Una telecamera di sicurezza che usa cloud inference smette di funzionare quando internet cade; una che usa TinyML continua.
- **Consumo:** trasmettere dati via radio è energeticamente costoso. Fare inference localmente ed inviare solo un byte di risultato può essere 100× più efficiente.
- **Costo:** eliminare il cloud compute riduce i costi operativi su scala (miliardi di dispositivi).
- **Always-on sensing:** alcuni task richiedono ascolto o sensing continuo (wake word detection, anomaly detection su vibrazioni) — impossibile tenere la radio WiFi sempre accesa.

I vincoli tipici dei target MCU su cui gira TinyML: RAM 256 KB (spesso meno), Flash 1–4 MB, clock da 16 MHz (Arduino UNO) a 240 MHz (ESP32), no FPU su molti device (solo aritmetica intera).

Tecniche di compressione modelli

I modelli ML standard sono troppo grandi per girare su MCU. Tre tecniche principali permettono di ridurre drasticamente dimensione e costo computazionale.

Quantizzazione

La **quantizzazione** riduce la precisione numerica dei pesi e delle attivazioni da floating point a 32 bit (FP32) a interi a 8 bit (INT8) o meno. L'intuizione è che la precisione a 32 bit è molto superiore a quella necessaria per l'inferenza — i pesi di una rete neurale sono stimati con errore di ottimizzazione già dell'ordine del 1%, quindi perdere qualche bit di precisione ha impatto minimo sull'accuracy.

Il mapping lineare da float a intero è: $x_q = \text{round}\left(\frac{x}{S}\right) + Z$ dove $S = \frac{x_{\max} - x_{\min}}{2^b - 1}$ è la scala e Z è il zero point (che permette di rappresentare lo zero esattamente in INT8).

Due modalità:

- **PTQ (Post-Training Quantization):** si quantizza un modello già addestrato in FP32. Semplice, richiede solo un piccolo dataset di calibrazione per stimare i range dei valori. Funziona bene per INT8, può degradare significativamente a INT4 o meno.
- **QAT (Quantization-Aware Training):** la quantizzazione viene simulata **durante** il training con «fake quantization nodes». Il modello impara a essere robusto alla riduzione di precisione. Produce modelli più accurati di PTQ, specialmente a basse bit-width (INT4 o meno), al costo di un training più lungo.

Benefici concreti: $4\times$ riduzione della dimensione del modello ($\text{FP32} \rightarrow \text{INT8}$), $2\text{--}4\times$ speedup sull'hardware che supporta operazioni INT8 (la maggior parte dei MCU lavora più velocemente in INT8 che in FP32).

Pruning

Il **pruning** rimuove dalla rete i pesi e le connessioni ridondanti — quelli il cui valore è vicino a zero contribuiscono poco al risultato finale. L'intuizione è che le reti neurali, dopo il training, sono tipicamente sovradimensionate: molti neuroni fanno poco. Rimuoverli riduce sia la dimensione del modello che il costo di inferenza.

Unstructured pruning: rimuove singoli pesi individualmente ($|w| < \text{soglia} \Rightarrow w = 0$). Può ottenere sparsità molto alte (90%+ dei pesi posti a zero) con perdita di accuracy minima, ma le matrici risultanti sono sparse — per accelerarle serve hardware o librerie che supportano sparse computation, altrimenti il guadagno in velocità è limitato.

Structured pruning: rimuove unità strutturali intere — interi filtri, canali, o layer. Il modello risultante è più piccolo ma rimane denso (senza sparse computation). Più facile da accelerare su hardware standard, ma meno flessibile del pruning non strutturato.

Il processo è tipicamente iterativo: Train $\rightarrow$ Prune $\rightarrow$ Fine-tune (per recuperare accuracy) $\rightarrow$ Ripeti con soglia più aggressiva.

Knowledge Distillation

La **knowledge distillation** è una tecnica per addestrare una rete piccola («student») a imitare il comportamento di una grande rete pre-addestrata («teacher»). La chiave è usare le **soft labels** — gli output del softmax del teacher invece delle label one-hot del ground truth. Le soft labels codificano le relazioni tra classi (es. il teacher assegna 70% a «gatto», 25% a «tigre», 5% a «leopardo») — questa informazione di struttura, chiamata «**dark knowledge**», non è presente nel ground truth ma aiuta enormemente lo student ad apprendere.

Il **temperature scaling** ammorbidisce la distribuzione del softmax per far emergere questa dark knowledge: $p_i = \frac{\exp(\frac{z_i}{T})}{\sum_j \exp(\frac{z_j}{T})}$ con $T > 1$. Con T alto, le probabilità sono più uniformi e le relazioni tra classi diventano più visibili.

La loss totale bilancia il segnale del teacher e il ground truth: $L = \alpha \cdot L_{\text{CE}(y, p_{\text{student}})} + (1 - \alpha) \cdot T^2 \cdot L_{\text{KL}}(p_{\text{teacher}}^T, p_{\text{student}}^T)$

Architetture efficienti

Oltre alla compressione post-hoc, esistono architetture progettate dall'inizio per essere efficienti.

MobileNets

L'idea chiave di MobileNet è la **Depthwise Separable Convolution**: fattorizza la convoluzione standard (costosa) in due operazioni più leggere. Prima una **depthwise convolution** applica un singolo filtro separatamente a ogni canale di input (cattura le feature spaziali all'interno di ogni canale); poi una **pointwise convolution** (conv 1×1) combina i canali (cattura le relazioni tra canali). Il risultato matematico è lo stesso di una convoluzione standard, ma il costo computazionale per una conv 3×3 si riduce di un fattore $8\text{--}9\times$. **MobileNetV2** aggiunge gli **inverted residual blocks** con bottleneck lineari per migliorare ulteriormente il tradeoff accuracy/efficienza.

EfficientNet

EfficientNet propone il **compound scaling**: scalare profondità, larghezza e risoluzione della rete insieme in modo coordinato con un unico coefficiente φ : $d = \alpha^\varphi$, $w = \beta^\varphi$, $r = \gamma^\varphi$, con il vincolo $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ che assicura che raddoppiare le FLOPs migliori il modello in modo bilanciato. L'architettura base EfficientNet-B0 viene trovata tramite **NAS (Neural Architecture Search)**.

Hardware per Edge AI

Diversi chip sono stati sviluppati specificamente per accelerare ML sul bordo:

- **Google Edge TPU**: ASIC ottimizzato per operazioni INT8, 4 TOPS (trillion operations per second) a soli 2W. Usato su Coral Dev Board.
- **Intel Movidius / Neural Compute Stick**: Vision Processing Unit in formato USB stick, 1+ TOPS, facilissimo da collegare a qualsiasi host (Raspberry Pi, PC).
- **ARM Cortex-M + CMSIS-NN**: framework di kernel ottimizzati per MCU ARM. Ultra-low power, nessun hardware ML dedicato, funziona su miliardi di dispositivi già prodotti.
- **NVIDIA Jetson** (Nano, TX2, AGX): GPU-based, molto più potente (da 0.5 a 32 TOPS), supporta PyTorch e TensorFlow completi. Ideale per edge computing più potente (robotica, visione artificiale).

Software

TFLite (TensorFlow Lite): runtime ML ottimizzato di Google per mobile e dispositivi edge. Supporta quantizzazione INT8 nativa e molti backend hardware (GPU, DSP, Edge TPU). **TFLite Micro**: sottoinsieme di TFLite per MCU con risorse minime — no allocazione dinamica di memoria, gira su bare metal senza OS, implementato in C++. **ONNX Runtime**: motore cross-platform di Microsoft che supporta modelli nel formato ONNX (interscambiabile tra PyTorch, TensorFlow, scikit-learn).

Edge Impulse

Edge Impulse è una piattaforma **end-to-end** per sviluppare, addestrare e deployare modelli ML su dispositivi embedded e edge. Copre l'intera pipeline senza richiedere expertise in ogni area: raccolta dati sul dispositivo → labeling e gestione dataset → progettazione e training del modello → ottimizzazione e deployment.

CLI Tools

- **edge-impulse-daemon**: connette il dispositivo alla piattaforma (sampling, flashing, testing da remoto)
- **edge-impulse-uploader**: carica dataset esistenti in formato CSV/WAV/JPG
- **edge-impulse-data-forwarder**: il più usato in lab — connette la board via **porta seriale**, legge i dati dal Serial Monitor e li invia automaticamente alla piattaforma. Non richiede modifica del firmware. Workflow: (1) board stampa dati su seriale, (2) esegui tool sul PC, (3) dati inviati in real-time alla dashboard.
- **edge-impulse-run-impulse**: esegue l'impulso addestrato direttamente sulla board connessa, mostrando i risultati in tempo reale

L'Impulso (pipeline)

Input data → Processing block (DSP) → Learning block (NN) → Output

- **Input block**: tipo di dato (serie temporale, immagine, audio) e finestra temporale

- **Processing block (DSP)**: estrae features — FFT, MFE, raw, ecc. Trasforma dati grezzi in **processed features**
- **Learning block**: rete neurale, K-NN, decision tree. Si addestra sulle processed features
- **Output block**: classi con probabilità

Alla creazione si specificano: **target device** (es. Cortex-M4F 80 MHz), **budget** di RAM/ROM/latenza. Edge Impulse stima latenza e memoria prima del deployment. Si possono creare più **esperimenti** sugli stessi dati con configurazioni diverse per trovare il miglior tradeoff.

Risultati del Training

Dopo il training: F1 score per classe, confusion matrix, feature explorer (clustering 2D/3D), stima performance sulla board target.

Model Optimization: Quantizzazione

Quantizzazione int8 vs float32 — dati reali dalle slide:

Versione	Latenza	RAM	Accuracy
Quantized (int8)	856 ms	283.1 KB	66.67%
Unoptimized (float32)	1287 ms	1.0 MB	100%

La versione quantizzata è **1.5× più veloce** e usa **3.5× meno RAM**, a costo di 33% di accuracy in questo caso. La scelta dipende dai requisiti dell'applicazione.

TFLite Workflow (dal modello al MCU)

1. Edge Impulse esporta il modello come file `.tflite`
2. `.tflite` → file header C: `unsigned char model_data[] = {0x1c, ...}`
3. Il file header viene incluso nel firmware Arduino/C++
4. TFLite Micro runtime carica il modello dall'array in RAM
5. Inferenza eseguita direttamente sul MCU

Deployment: Arduino library (.zip), C++ library, binary compilato; scelta tra versione ottimizzata (int8) e non (float32).

Federated Learning

Il **Federated Learning** affronta un problema fondamentale del ML in ambito IoT e mobile: come addestrare un modello su dati distribuiti su milioni di dispositivi, rispettando la privacy degli utenti e i vincoli di banda? La risposta è semplice ma potente: invece di centralizzare i dati, si distribuisce il modello.

Il modello standard è **FedAvg** (Federated Averaging): il server inizializza il modello globale w_0 . Ad ogni round di training, il server seleziona K client (dispositivi) tra quelli disponibili, invia il modello corrente w_t a tutti, ogni client addestra localmente per E epoche sui propri dati privati (senza mai trasmetterli), poi invia al server solo l'aggiornamento del modello (i gradienti o i nuovi pesi). Il server aggrega gli aggiornamenti con una media pesata per la dimensione del dataset: $w_{t+1} = \sum_k \left(\frac{n_k}{n}\right) \cdot w_k^{t+1}$, dove n_k è il numero di campioni del client k e n è il totale. Il modello aggregato viene ridistribuito per il round successivo.

Le sfide pratiche sono significative: i dati reali su dispositivi diversi sono tipicamente **non-IID** (non identicamente distribuiti) — l'utente A e l'utente B hanno pattern molto diversi, il che rallenta la

convergenza rispetto al training centralizzato. La **comunicazione** è costosa (trasmettere l'intero modello ad ogni round può essere problematico per modelli grandi). I **stragglers** (dispositivi lenti che non completano l'aggiornamento in tempo) devono essere gestiti aspettando o ignorandoli. E la privacy non è garantita al 100%: da un aggiornamento del modello è possibile in teoria ricostruire parzialmente i dati. Le soluzioni includono la **differential privacy** (aggiungere rumore calibrato agli aggiornamenti per dare garanzie formali di privacy) e la **secure aggregation** (protocollo crittografico che permette al server di calcolare la somma degli aggiornamenti senza vedere nessun aggiornamento individuale).

Split Computing

Il **Split Computing** è una variante del task offloading applicata specificamente alle reti neurali. L'idea è partizionare la rete neurale in due parti: i **primi layer** girano on-device (feature extraction di basso livello), le **feature intermedie** vengono trasmesse all'edge/cloud via rete, e i **layer rimanenti** completano l'inferenza sull'edge.

Il vantaggio rispetto all'offloading completo è che le feature intermedie sono spesso molto più compatte dell'input grezzo: un'immagine da 100KB può produrre feature intermedie di pochi KB dopo i primi layer di convoluzione. Quindi si trasmette meno rispetto al full offloading. Il vantaggio rispetto all'inferenza locale è che i layer profondi (più costosi computazionalmente) vengono eseguiti sull'edge con risorse abbondanti.

Bottleneck injection: per comprimere ulteriormente le feature intermedie al punto di split, si inserisce un layer autoencoder che comprime le feature prima della trasmissione e le ricostruisce sull'edge. Questo crea un tradeoff esplicito tra compressione (risparmio di banda) e perdita di informazione (degradazione dell'accuracy).

Early Exit

L'**Early Exit** sfrutta il fatto che campioni diversi richiedono computazioni di complessità diversa: un'immagine di un gatto in condizioni ottimali è classificabile già con pochi layer; un'immagine ambigua richiede tutta la profondità della rete.

Il meccanismo consiste nell'attaccare classificatori ausiliari (tipicamente piccole reti lineari) a varie profondità della rete principale. Durante l'inferenza, dopo ogni classificatore si verifica la confidenza: se supera una soglia, si restituisce la predizione corrente senza calcolare i layer più profondi. Se non supera la soglia, si continua verso i layer più profondi.

Il risultato è una riduzione significativa del **tempo medio di inference**: i campioni «facili» escono presto (un layer o due), quelli difficili percorrono tutta la rete. Questo è particolarmente utile in applicazioni real-time dove il caso medio è più rilevante del caso peggiore.

Node-RED

Cos'è Node-RED

Node-RED è uno strumento di sviluppo visuale per collegare device, API e servizi online in un sistema IoT integrato, senza scrivere (quasi) nessun codice. È costruito su **Node.js** e si basa sul paradigma di **programmazione flow-based**: si trascinano nodi sul canvas, li si collega con fili, e si clicca Deploy. Il risultato è un'applicazione funzionante.

Perché Node-RED è così popolare nell'IoT: **sviluppo rapido** (in 10 minuti si può collegare un sensore MQTT a un database e una dashboard), supporto **out-of-the-box** per i protocolli IoT più usati (MQTT, HTTP/REST, WebSocket, CoAP), **comunità enorme** con migliaia di nodi pronti da installare per integrare qualsiasi servizio (InfluxDB, Telegram, Twitter, GPIO, ecc.), e **estensibilità** — chiunque può scrivere nodi custom in JavaScript.

Node-RED nell'architettura IoT

Node-RED è abbastanza flessibile da girare sia all'edge (su un Raspberry Pi vicino ai sensori) che nel cloud (su un server). Nella architettura di riferimento Hitachi, Node-RED copre sette ruoli tipici in un sistema IoT completo: edge analytics (elabora i dati vicino alla sorgente), data collection via MQTT (raccolge i messaggi dai sensori), device control (invia comandi agli attuatori), dashboard (visualizza i dati in tempo reale), data accumulation (scrive su database come InfluxDB), integration con sistemi esistenti tramite REST, e connessione a servizi esterni.

Concetti fondamentali

Nodi

Il concetto base di Node-RED è il **nodo**: un box con una logica specifica che riceve messaggi, li elabora, e produce messaggi in output. Ogni nodo ha zero o più porte di input (a sinistra) e zero o più porte di output (a destra). Si connettono tra loro trascinando fili dalle porte di output alle porte di input.

Tre categorie:

- **Input nodes** (es. `inject`, `mqtt in`, `http in`): hanno solo porte di output. Sono le sorgenti del flow — si attivano su eventi esterni (un messaggio MQTT arriva, un timer scatta, una richiesta HTTP arriva) e iniettano messaggi nel flow.
- **Output nodes** (es. `debug`, `mqtt out`, `file`): hanno solo porte di input. Sono i terminali del flow — ricevono messaggi e producono effetti nel mondo (scrivono su un file, pubblicano su MQTT, mandano una risposta HTTP).
- **Processing nodes** (es. `function`, `switch`, `change`, `delay`): hanno sia input che output. Trasformano, filtrano, instradano o modificano i messaggi che ricevono.

Flows e Deploy

I nodi sono organizzati in **flow**, ognuno su una tab separata del canvas. Si possono avere quanti flow si vuole — tipicamente uno per funzionalità logica (es. «gestione sensori», «dashboard», «allarmi»). Quando si clicca il pulsante **Deploy**, Node-RED compila tutti i flow e li mette in esecuzione immediatamente. Le modifiche sono operative in pochi secondi.

Messaggi e `msg.payload`

Il meccanismo di comunicazione tra nodi sono i **messaggi** JavaScript — oggetti con proprietà arbitrarie. La convenzione è che la proprietà principale di interesse si chiami `msg.payload` e contenga il dato rilevante (un numero, una stringa, un oggetto JSON). I nodi possono aggiungere altre proprietà (es. `msg.topic` per il topic MQTT, `msg.headers` per gli header HTTP) che i nodi successivi possono leggere. La best practice è sempre inserire un nodo `debug` per ispezionare la struttura del messaggio prima di fare assunzioni sulla sua forma.

Nodi principali

Nodo	Funzione
<code>inject</code>	Inietta un messaggio nel flow (manualmente o a intervalli)
<code>debug</code>	Mostra proprietà dei messaggi nel debug sidebar
<code>function</code>	Funzione JavaScript eseguita sui messaggi ricevuti
<code>switch</code>	Instrada messaggi in base a valori delle proprietà
<code>change</code>	Modifica proprietà dei messaggi (set, delete, move)
<code>delay</code>	Ritarda messaggi o limita il rate
<code>mqtt in</code>	Connette a broker MQTT e sottoscrive a un topic
<code>mqtt out</code>	Pubblica messaggi su broker MQTT
<code>http request</code>	Invia richieste HTTP e restituisce la risposta
<code>json</code>	Converte tra stringa JSON e oggetto JavaScript
<code>write file</code>	Scrive <code>msg.payload</code> su file (append o replace)

Dashboard

Estensione `node-red-dashboard` per creare dashboard direttamente dentro Node-RED. Widget: button, dropdown, switch, slider, numeric, text input, date picker, gauge, chart, audio out, notification.

Context Storage

Tre livelli di contesto:

- **Node context**: visibile solo al nodo
- **Flow context**: condiviso tra nodi dello stesso flow
- **Global context**: condiviso tra tutti i flow

Privacy nell'IoT

Il problema

La privacy nell'IoT non è un problema teorico: è un problema concreto e urgente che i dati di campo rendono evidente. Uno studio HP su 10 dispositivi IoT consumer ha trovato risultati allarmanti: l'**80%** sollevava preoccupazioni privacy e non richiedeva password forti, il **70%** trasmetteva la comunicazione in chiaro senza crittografia, il **60%** aveva problemi con le interfacce utente. La «password sicura» più diffusa? Tutti i dispositivi testati accettavano «123456».

Le conseguenze reali vanno oltre il fastidio: auto IoT hackerate da remoto mentre erano in marcia (ricercatori Chrysler, 2015), spam inviato da frigoriferi e smart TV compromessi, la **Mirai Botnet** del 2016 che ha usato 600.000 dispositivi IoT (telecamere di sorveglianza, router, DVR) con credenziali di default per lanciare il più grande attacco DDoS della storia (1.2 Tbps), baby monitor con video trasmessi pubblicamente su internet, attacchi a dispositivi medici ospedalieri, e — il caso più tragicomico — il falso allarme missilistico delle Hawaii del 2018, causato in parte da una password scritta su un post-it visibile in una fotografia pubblica.

Le 7 Privacy Threats (Bedogni)

I rischi per la privacy nell'IoT si articolano in sette categorie distinte, ognuna con la propria natura e le proprie soluzioni:

1. **Identification:** il rischio che dai dati raccolti si possa risalire all'identità del proprietario. Più sottile di quanto sembri: Latanya Sweeney ha dimostrato che l'87% dei cittadini USA è univocamente identificabile dalla sola combinazione di ZIP code + data di nascita + sesso — tre dati apparentemente innocui che da soli non identificano nessuno, ma combinati lo fanno con alta probabilità.
2. **Tracking:** costruire la storia dei movimenti di un individuo nel tempo e nello spazio. Richiede qualche forma di identificazione (anche pseudonima) per correlare osservazioni successive alla stessa persona. I pattern di movimento sono quasi un'impronta digitale comportamentale.
3. **User Profiling:** dedurre interessi, abitudini, stato di salute, orientamento politico o religioso di un individuo correlando i suoi dati con altri dataset. La contromisura principale è lo splitting e la distribuzione dei dati — se nessuna entità ha accesso all'intero profilo, è più difficile costruirlo.
4. **Interaction and Presentation:** il rischio di rivelare dati a listener non intenzionali — ad esempio una schermata con dati medici visibile a terze persone, o dati trasmessi in chiaro intercettabili sulla rete. Gestito con crittografia e controllo degli accessi fisici e logici.
5. **Lifecycle transitions:** i dispositivi cambiano proprietario — vengono venduti, donati, dismessi. Un dispositivo smart con storia di dati personali (posizioni, abitudini, credenziali) diventa una minaccia quando passa in mani nuove senza un reset completo. La soluzione è avere meccanismi di factory reset affidabili e la politica di reset prima di qualsiasi trasferimento.
6. **Inventory attacks:** attacchi mirati a rubare l'inventario dei dati presenti su un dispositivo o sistema. La difesa è minimizzare i dati memorizzati (non tenere quello che non serve) e proteggere l'accesso con autenticazione forte.

7. **Linkage**: correlare dati apparentemente separati e innocui per ricavare informazioni più sensibili. Esempio classico: un dataset contiene pseudoidentificatore + GPS; un altro dataset contiene pseudoidentificatore + nome reale. Combinandoli, la pseudoanonimizzazione del primo diventa inutile. Questo è il motivo per cui la semplice pseudoanonimizzazione non è sufficiente.

Classificazione dati

Non tutti i dati hanno lo stesso livello di sensitività, e la classificazione è il primo passo per gestirli correttamente. Quattro categorie:

- **Sensitive**: messaggi privati, posizione GPS precisa, preferenze politiche/religiose, dati biometrici. Richiedono le protezioni più stringenti.
- **Personal**: identificatori utente/device, nome, email, indirizzo IP. Identificano ma non necessariamente rivelano aspetti sensibili della vita.
- **Non-sensitive**: dati aggregati statistici, dati che non possono essere linkati a un individuo. Non richiedono protezioni speciali.
- **Uncertain**: condizione medica, data di nascita, etnia. Sensibili in alcuni contesti, meno in altri.

La sensitività non è additiva in modo semplice: la regola base è che il dato più sensibile determina la sensitività del set (red + green = red). Ma l'aggregazione può **diminuire** la sensitività: la temperatura media di un quartiere non rivela nulla sull'individuo, anche se la temperature di ogni singola casa lo farebbe. E può **aumentarla inaspettatamente**: quattro dati individualmente innocui (orario di sveglia + destinazione di lavoro + orario di ritorno + consumo elettrico serale) combinati rivelano se qualcuno vive solo e quando è assente di casa.

Privacy by Design (7 principi Cavoukian)

Ann Cavoukian, ex Information and Privacy Commissioner dell'Ontario, ha formalizzato il concetto di «Privacy by Design» — l'idea che la privacy non debba essere un add-on aggiunto dopo, ma un requisito di progetto dal primo giorno:

1. **Data minimization**: raccogliere e memorizzare solo il dato strettamente necessario per il task. Se non hai bisogno del numero di telefono, non raccoglierlo. Meno dati = meno rischi.
2. **Controllability**: il proprietario dei dati deve avere controllo reale su come sono usati — non solo in teoria, ma con strumenti pratici (cancellazione, export, revoca del consenso).
3. **Transparency**: l'utente deve essere notificato su quali dati vengono raccolti, come, perché, e con chi sono condivisi — in modo comprensibile, non in legalese di 50 pagine.
4. **User friendly systems**: la privacy non deve essere un ostacolo tecnico per l'utente. I settings di privacy devono essere accessibili e comprensibili, non nascosti in menu inesplicabili.
5. **Data confidentiality**: i dati devono rimanere riservati — crittografia in transito, crittografia a riposo, controllo degli accessi.
6. **Data quality**: tenere solo dati accurati e aggiornati. Dati errati non sono solo inutili, possono causare danni (es. un indirizzo sbagliato in un sistema di emergenza).
7. **Use limitation**: i dati raccolti per uno scopo non devono essere usati per altri scopi. Il dato sulla posizione raccolto per la navigazione non deve essere usato per il profiling pubblicitario.

GDPR e il contesto normativo europeo

Il **GDPR** (General Data Protection Regulation, Reg. UE 2016/679) è la normativa cardine per la protezione dei dati personali nell'Unione Europea, entrato in vigore nel 2018. È particolarmente rilevante per i sistemi IoT, che per natura raccolgono dati personali in modo continuo e pervasivo. I quattro pilastri principali:

- **Consenso esplicito:** i dati personali possono essere raccolti solo con consenso chiaro e informato dell'utente — non pre-spuntato, non nascosto nei T&C.
- **Diritto all'oblio:** l'utente può richiedere la cancellazione di tutti i propri dati.
- **Portabilità:** l'utente ha il diritto di ricevere i propri dati in formato leggibile da macchina e trasferirli a un altro fornitore.
- **Notifica delle violazioni:** in caso di data breach, l'organizzazione deve notificare l'autorità garante entro 72 ore e, se il rischio è elevato, anche gli interessati direttamente.

Il GDPR si affianca al **Cyber Resilience Act** (CRA) come strumento normativo europeo: mentre il CRA si concentra sulla sicurezza del prodotto durante il suo intero ciclo di vita, il GDPR tutela i dati personali degli utenti che quei prodotti raccolgono.

PETs (Privacy Enhancing Technologies) — 8 strategie

Le PETs sono le tecniche concrete che implementano i principi di Privacy by Design:

1. **Minimize:** raccogliere la minima quantità di dati necessaria. Anonimizzare o pseudo-anonimizzare i dati alla sorgente.
2. **Hide:** nascondere i dati attraverso crittografia, onion routing (Tor), credenziali anonime che provano attributi senza rivelare l'identità.
3. **Split:** distribuire i dati su più storage in modo che nessuna entità abbia un profilo completo. La combinazione non deve essere possibile senza autorizzazione esplicita.
4. **Merge:** aggregare i dati a un livello di granularità che non permette l'identificazione individuale. Pubblicare statistiche di gruppo invece di dati individuali.
5. **Notify:** essere trasparenti in modo proattivo — informare gli utenti dei dati raccolti, dei possibili leak, e degli incidenti di sicurezza. Non aspettare che lo scoprano da soli.
6. **Control:** garantire il consenso informato reale dell'utente, con possibilità di revoca. L'«accettare tutto» di default non è consenso.
7. **Enforce:** controllare tecnicamente chi può accedere ai dati (autenticazione, autorizzazione, auditing).
8. **Log:** tenere un audit trail di quali dati sono stati registrati, chi vi ha acceduto, e quali modifiche sono state apportate. Essenziale per rilevare accessi non autorizzati e per la compliance normativa.

Usable Privacy

Un problema spesso trascurato è che anche la migliore privacy tecnologica diventa inutile se gli utenti non riescono a configurarla o capirla. I meccanismi tradizionali di configurazione della privacy hanno tutti gravi limiti pratici:

- **Settings:** troppo complessi per utenti non tecnici. Decine di toggles con nomi oscuri che nessuno capisce.
- **Policies:** troppo lunghe (una ricerca ha stimato che leggere tutte le privacy policy a cui si è esposti richiederebbe 76 giorni all'anno) e scritte in linguaggio legale inaccessibile.
- **NDA:** completamente system-centered — proteggono l'organizzazione, non l'utente.
- **Terms & Conditions:** troppo tecnici e lunghi. Il 99% degli utenti fa «I agree» senza leggere.

L'insight chiave della ricerca in usable privacy è che invece di cercare di far leggere tutti quei documenti, si dovrebbe **focalizzarsi su ciò che conta davvero per l'utente** e comunicarlo in modo immediato, visivo e contestuale — notifiche just-in-time quando un dato viene usato, icone standardizzate, controlli semplificati.

OWASP IoT Top 10 (2018)

L'OWASP IoT Top 10 è la lista delle vulnerabilità più critiche nei sistemi IoT, una reference fondamentale per chi progetta o valuta sicurezza IoT:

1. Weak, guessable, or hardcoded passwords — credenziali di default mai cambiate o hard-coded nel firmware
2. Insecure network services — servizi di rete non necessari esposti, servizi senza autenticazione
3. Insecure ecosystem interfaces — API web, app mobile o cloud senza adeguata autenticazione/autorizzazione
4. Lack of secure update mechanism — aggiornamenti firmware senza verifica di firma, o assenza di meccanismo di update
5. Use of insecure or outdated components — librerie di terze parti con vulnerabilità note, OS non aggiornati
6. Insufficient privacy protection — raccolta di dati personali senza necessità, storage non cifrato
7. Insecure data transfer and storage — comunicazioni in chiaro, credenziali in chiaro nel codice o nel log
8. Lack of device management — nessun modo di disabilitare o de-registrare device compromessi
9. Insecure default settings — configurazione di default insicura (telnet abilitato, UPnP esposto, password vuota)
10. Lack of physical hardening — accesso fisico non protetto (porte debug esposte, JTAG accessibile, nessuna cifratura del filesystem)

Cyber Resilience Act (EU 2024/2847)

Il **Cyber Resilience Act** è il regolamento europeo che per la prima volta impone **requisiti obbligatori di cybersecurity** per tutti i prodotti con componenti digitali venduti nell'UE — dai router domestici ai dispositivi medici, dalle telecamere di sorveglianza ai toy connessi. I fabbricanti devono dimostrare conformità, mantenere aggiornamenti di sicurezza per tutta la vita del prodotto, e notificare le vulnerabilità alle autorità competenti. Per i dati di aziende UE gestiti fuori dall'UE si applicano comunque i principi del regolamento — il territorio fisico del dato non determina la giurisdizione applicabile.

